

EXPERIMENT- 1

1. Experiment Title: Demonstrate inline, internal, and external JavaScript; use console methods; create a webpage showing user information and a welcome message.

2. Software/Tools Required: VS Code , Google Chrome

3. Experiment Program Code :

External CSS.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Experiment 1</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div>
    <h1>Welcome to the Lab!</h1>
    <p><strong>Name:</strong> Ashlesha Singh</p>
    <p><strong>PRN:</strong> 24070521221</p>
    <button onclick="alert('External CSS active!')">Test External CSS</button>
  </div>
</body>
</html>
```

style.css (External CSS)

```
/* External CSS used to style the page */
body {
  font-family: 'Segoe UI', sans-serif;
  background-color: #f4f4f9;
  display: flex;
  justify-content: center;
  padding-top: 50px;
}

div {
  background-color: white;
  padding: 40px;
  border-radius: 8px;
  border-top: 5px solid blue; /* Adds a nice accent */
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
```

```

        text-align: center;
        width: 350px;
    }

    h1 {
        color: #333;
        font-size: 22px;
    }

    button {
        padding: 10px 20px;
        background-color: blue;
        color: white;
        border: none;
        border-radius: 4px;
        cursor: pointer;
        font-size: 14px;
    }

    button:hover {
        background-color: darkblue;
    }

```

Inline CSS.html

```

<!DOCTYPE html>
<html>
<head>
    <title>Experiment 1</title>
</head>
<body style="font-family: 'Segoe UI', sans-serif; background-color: #f4f4f9; display:
flex; justify-content: center; padding-top: 50px; margin: 0;">

    <div style="background-color: white; padding: 40px; border-radius: 8px; border-top:
5px solid blue; box-shadow: 0 2px 10px rgba(0,0,0,0.1); text-align: center; width:
350px;">

        <h1 style="color: #333; font-size: 22px;">Welcome to the Lab!</h1>

        <p><strong>Name:</strong> Ashlesha Singh</p>
        <p><strong>PRN:</strong> 24070521221</p>

        <button onclick="alert('Inline CSS active!')"
            style="padding: 10px 20px; background-color: blue; color: white; border: none;
border-radius: 4px; cursor: pointer; font-size: 14px;">

```

```
        Test Inline CSS
    </button>
</div>
</body>
</html>
```

Internal CSS.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Experiment 1</title>

    <style>
        body {
            font-family: 'Segoe UI', sans-serif;
            background-color: #f4f4f9;
            display: flex;
            justify-content: center;
            padding-top: 50px;
            margin: 0;
        }

        div {
            background-color: white;
            padding: 40px;
            border-radius: 8px;
            border-top: 5px solid blue;
            box-shadow: 0 2px 10px rgba(0,0,0,0.1);
            text-align: center;
            width: 350px;
        }

        h1 {
            color: #333;
            font-size: 22px;
        }

        button {
            padding: 10px 20px;
            background-color: blue;
            color: white;
            border: none;
            border-radius: 4px;
```

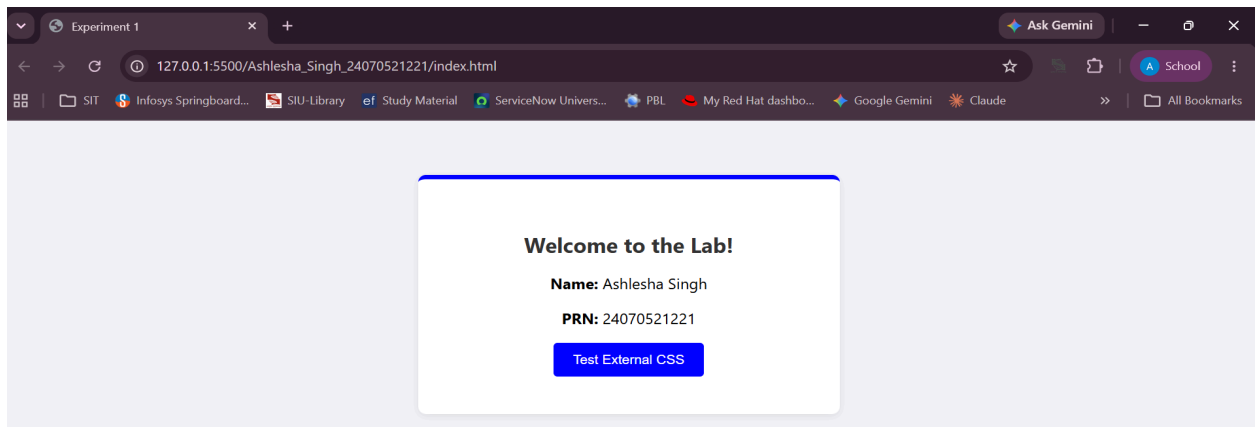
```
        cursor: pointer;
        font-size: 14px;
    }

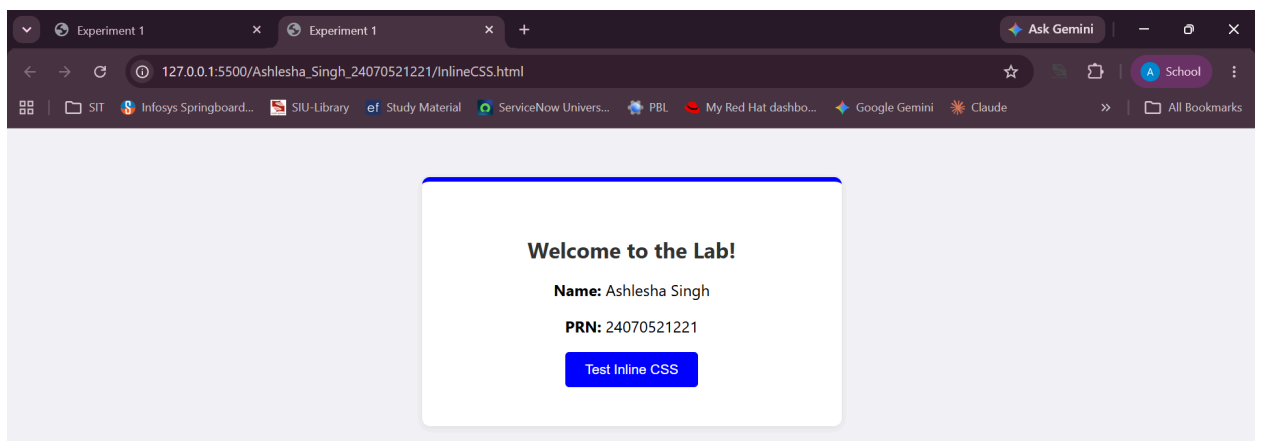
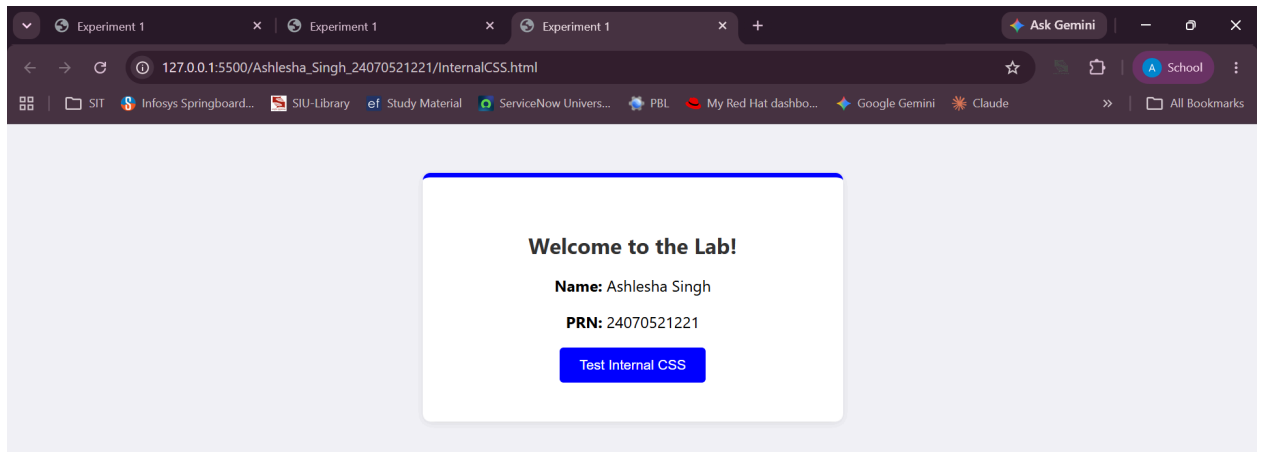
    button:hover {
        background-color: darkblue;
    }
</style>
</head>
<body>

<div>
    <h1>Welcome to the Lab!</h1>
    <p><strong>Name:</strong> Ashlesha Singh</p>
    <p><strong>PRN:</strong> 24070521221</p>
    <button onclick="alert('Internal CSS active!')">Test Internal CSS</button>
</div>

</body>
</html>
```

4.Output:





5. Result/Conclusion: The experiment was successfully performed by implementing JavaScript through three different integration methods. Console functions were verified using browser developer tools, and the required environment information was displayed on the webpage.

CASE STUDY - 1

1. Case Study Title: Create a webpage for the college department and link the student information page with it and write a Java Script Program to Display “ Welcome to SIT Nagpur ” in an alert Box.

2. Software/Tools Required: VS Code , Google Chrome

3. Case Study Program Code:

CS1_Main.html

```
<html>

<head>

<title>SIT Nagpur - Department Page</title>

<style>

    body {

        font-family: Arial, sans-serif;

        margin: 40px;

        background-color: #f4f4f9;

        color: #333;

    }

    .container {

        max-width: 600px;

        background: #fff;

        padding: 25px;

        border-radius: 8px;

        box-shadow: 0 2px 5px rgba(0,0,0,0.1);

    }
```

```
h1 {
    color: #1a365d;
}

.btn {
    display: inline-block;
    margin-top: 15px;
    padding: 10px 15px;
    background-color: #3182ce;
    color: #fff;
    text-decoration: none;
    border-radius: 5px;
}

.btn:hover {
    background-color: #2b6cb0;
}

</style>

</head>

<body>

    <div class="container">

        <h1>Department of Computer Science & Engineering</h1>

        <p>Welcome to the official department portal of SIT Nagpur. We offer
state-of-the-art facilities and top-tier academic guidance to shape the innovators of
tomorrow.</p>

        <!-- Linking the student information page -->
```

```
<a href="CS1_Stud.html" class="btn">View Student Information</a>
</div>
```

```
<!-- JavaScript Program to display the Alert Box -->
```

```
<script>
    window.onload = function() {
        alert("Welcome to SIT Nagpur");
    };
</script>
```

```
</body>
```

```
</html>
```

CS1_Stud.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Information</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 40px;
            background-color: #f4f4f9;
            color: #333;
        }
        .container {
```

```
    max-width: 600px;
    background: #fff;
    padding: 25px;
    border-radius: 8px;
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}

h2 {
    color: #1a365d;
}

table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 15px;
}

th, td {
    padding: 10px;
    border: 1px solid #ddd;
    text-align: left;
}

th {
    background-color: #edf2f7;
}

.back-link {
    display: inline-block;
    margin-top: 20px;
```

```
        color: #3182ce;

        text-decoration: none;
    }

    .back-link:hover {

        text-decoration: underline;
    }
</style>
</head>
<body>

<div class="container">

    <h2>Student Information Page</h2>

    <table>

        <thead>

            <tr>

                <th>Roll No</th>

                <th>Student Name</th>

                <th>Year</th>

            </tr>

        </thead>

        <tbody>

            <tr>

                <td>24070521221</td>

                <td>Ashlesha Singh</td>
```

```

        <td>3rd Year</td>

    </tr>

</tbody>

</table>

<!-- Link back to Department page -->

    <a href="CS1_Main.html" class="back-link">← Back to Department
Page</a>

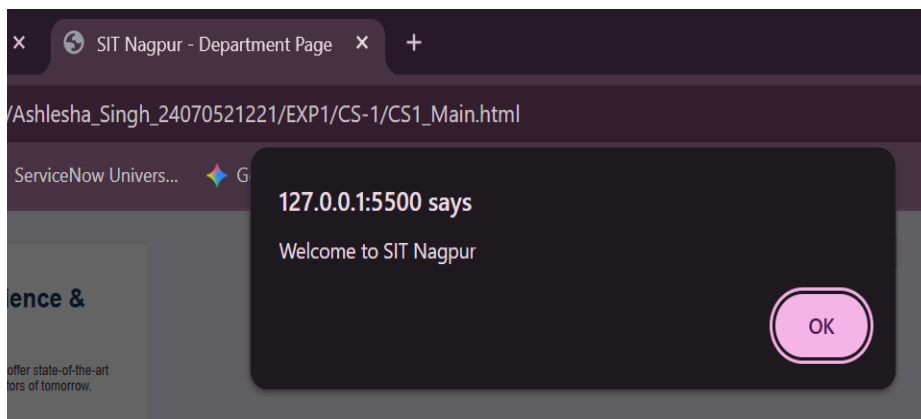
</div>

</body>

</html>

```

4. Output:



Department of Computer Science & Engineering

Welcome to the official department portal of SIT Nagpur. We offer state-of-the-art facilities and top-tier academic guidance to shape the innovators of tomorrow.

[View Student Information](#)

Student Information Page

Roll No	Student Name	Year
24070521221	Ashlesha Singh	3rd Year

[← Back to Department Page](#)

5. Result/Conclusion: In conclusion, this case study successfully demonstrates the design and development of a functional college department webpage integrated with a linked student information portal. By leveraging HTML for structure, CSS for presentation, and JavaScript for dynamic interactivity, the project effectively fulfills its technical objectives. The successful execution of the JavaScript alert box displaying "Welcome to SIT Nagpur" ensures an interactive and engaging user experience upon accessing the site. Ultimately, this implementation provides a streamlined, accessible, and professional web interface that serves as a practical blueprint for institutional information management.

EXPERIMENT- 2

1. Experiment Title: Use var, let, const, and demonstrate basic features like template literals and destructuring; build a billing calculator with user input.

2. Software/Tools Required: VS Code , Google Chrome

3. Experiment Program Code :

```
<!DOCTYPE html>

<html>

<head>

    <title>Billing Calculator</title>

</head>

<body>


    <h2>Billing Calculator</h2>


    <script>


const GST = 0.18;


var customerName = prompt("Enter Customer Name: ");
var productName = prompt("Enter Product Name: ");


var quantity = Number(prompt("Enter quantity"));
let price = Number(prompt("Enter price per Item"));
```

```
let total = quantity * price;  
let gstAmount = total * GST;  
let finalBill = total + gstAmount;
```

```
const bill = {  
  customer: customerName,  
  product: productName,  
  quantity,  
  price,  
  total,  
  gstAmount,  
  finalBill  
};
```

```
const {  
  customer,  
  product,  
  gstAmount: gst,  
  finalBill: amount  
} = bill;
```

```
document.write(`<h3>Billing Receipt</h3>`);  
document.write(`<p>Customer: ${customer}</p>`);  
document.write(`<p>Product: ${product}</p>`);  
document.write(`<p>Quantity: ${quantity}</p>`);
```

```
document.write(`<p>Price per Item: ₹${price}</p>`);  
document.write(`<p>Total: ₹${total}</p>`);  
document.write(`<p>GST (18%): ₹${gst.toFixed(2)}</p>`);  
document.write(`<h3>Final Bill: ₹${amount.toFixed(2)}</h3>`);  
  
</script>  
</body>  
</html>
```

4.Output:

127.0.0.1:5500 says

Enter Customer Name:

OK Cancel

127.0.0.1:5500 says

Enter Product Name:

OK Cancel

127.0.0.1:5500 says

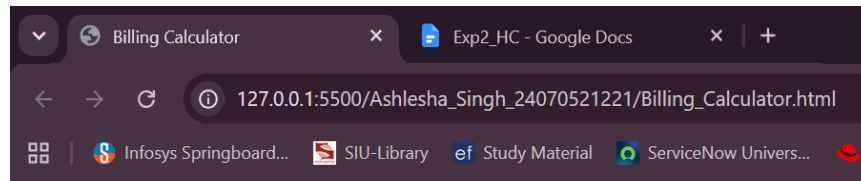
Enter quantity

OK Cancel

127.0.0.1:5500 says

Enter price per Item

OK Cancel



Billing Calculator

Billing Receipt

Customer: Ashlesha Singh - 24070521221

Product: Ashlesha Dell Laptop-221

Quantity: 221

Price per Item: ₹221

Total: ₹48841

GST (18%): ₹8791.38

Final Bill: ₹57632.38

5.Result/Conclusion:The HTML and JavaScript-based billing calculator was successfully implemented and executed. The program successfully accepts the customer name, product name, quantity, and price via user prompts, calculates the subtotal, applies an 18% GST, and dynamically renders a formatted billing receipt directly onto the web page.

CASE STUDY - 2

1. Case Study Title: Develop the webpage for an online shopping website using HTML,CSS and HTML5 tags And calculate the bill.

- Customer Name
- Mobile Number
- Invoice Number
- Product Name
- Quantity (in Kg)
- Rate per Kg (₹)
- Discount (%)
- GST (%)
- Packing Charges (₹)
- Payment Mode (Cash/Card/UPI)
- Membership (Yes/No)

2. Software/Tools Required: VS Code , Google Chrome

3. Case Study Program Code:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Online Shopping Billing Calculator</title>

  <style>

    body {

      font-family: Arial, sans-serif;

      margin: 20px;

      background-color: #f4f4f9;
```

```
}  
  
.container {  
    max-width: 500px;  
    background: #ffffff;  
    padding: 20px;  
    border-radius: 8px;  
    box-shadow: 0px 0px 10px rgba(0, 0, 0, 0.1);  
}  
  
h2, h3 {  
    color: #333;  
}  
  
p {  
    margin: 6px 0;  
    font-size: 15px;  
}  
  
hr {  
    border: 0;  
    height: 1px;  
    background: #ccc;  
    margin: 15px 0;  
}  
  
.final-amount {  
    color: #2e7d32;  
    font-size: 18px;  
    font-weight: bold;
```

```
    }
</style>
</head>
<body>

<div class="container">

    <h2>Online Shopping Billing Calculator</h2>

    <div id="receipt"></div>

</div>

<script>

    // 1. Collecting Inputs as specified in the Case Study

    var customerName = prompt("Enter Customer Name:");
    var mobileNumber = prompt("Enter Mobile Number:");
    var invoiceNumber = prompt("Enter Invoice Number:");
    var productName = prompt("Enter Product Name:");

    var quantity = Number(prompt("Enter Quantity (in Kg):"));
    var ratePerKg = Number(prompt("Enter Rate per Kg (₹):"));
    var discountPercent = Number(prompt("Enter Discount (%):"));
    var gstPercent = Number(prompt("Enter GST (%):"));
    var packingCharges = Number(prompt("Enter Packing Charges (₹):"));

    var paymentMode = prompt("Enter Payment Mode (Cash/Card/UPI:");
    var isMember = prompt("Membership Status (Yes/No):");
```

```
// 2. Calculations
```

```
let baseAmount = quantity * ratePerKg;
```

```
let discountAmount = (baseAmount * discountPercent) / 100;
```

```
let amountAfterDiscount = baseAmount - discountAmount;
```

```
let gstAmount = (amountAfterDiscount * gstPercent) / 100;
```

```
let finalBill = amountAfterDiscount + gstAmount + packingCharges;
```

```
// 3. Storing in an Object
```

```
const bill = {
```

```
  customerName,
```

```
  mobileNumber,
```

```
  invoiceNumber,
```

```
  productName,
```

```
  quantity,
```

```
  ratePerKg,
```

```
  discountPercent,
```

```
  discountAmount,
```

```
  gstPercent,
```

```
  gstAmount,
```

```
  packingCharges,
```

```
  paymentMode,
```

```
  isMember,
```

```
  finalBill
```

```
};
```

```
// 4. Destructuring Object for Output
```

```
const {  
    customerName: cName,  
    mobileNumber: mNo,  
    invoiceNumber: invNo,  
    productName: pName,  
    quantity: qty,  
    ratePerKg: rate,  
    discountAmount: disc,  
    gstAmount: gst,  
    packingCharges: packing,  
    paymentMode: mode,  
    isMember: member,  
    finalBill: total  
} = bill;
```

```
// 5. Displaying Results on Page
```

```
document.getElementById("receipt").innerHTML = `  
    <h3>Invoice Receipt</h3>  
    <p><strong>Invoice No:</strong> ${invNo}</p>  
    <p><strong>Customer Name:</strong> ${cName}</p>  
    <p><strong>Mobile No:</strong> ${mNo}</p>  
    <p><strong>Membership Status:</strong> ${member}</p>
```

```

<hr>

<p><strong>Product Name:</strong> ${pName}</p>

<p><strong>Quantity:</strong> ${qty} Kg</p>

<p><strong>Rate per Kg:</strong> ₹${rate.toFixed(2)}</p>

<p><strong>Base Amount:</strong> ₹${baseAmount.toFixed(2)}</p>

<p><strong>Discount    (${discountPercent}%):</strong>
-₹${disc.toFixed(2)}</p>

<p><strong>GST (${gstPercent}%):</strong> +₹${gst.toFixed(2)}</p>

<p><strong>Packing Charges:</strong> +₹${packing.toFixed(2)}</p>

<hr>

<p><strong>Payment Mode:</strong> ${mode}</p>

<p class="final-amount">Final Bill Amount: ₹${total.toFixed(2)}</p>

`;

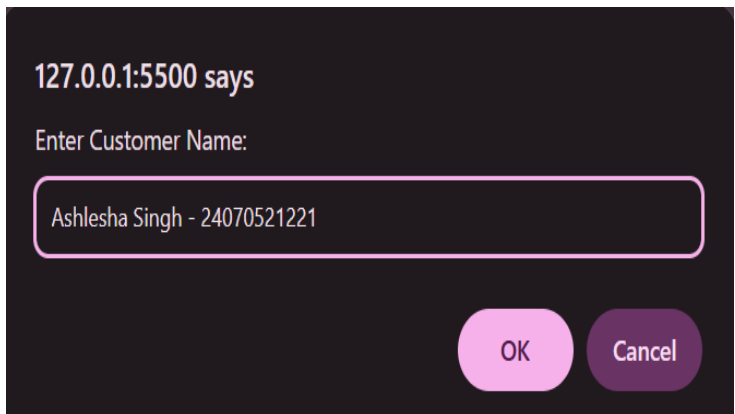
</script>

</body>

</html>

```

4. Output:



127.0.0.1:5500 says

Enter Customer Name:

Ashlesha Singh - 24070521221

OK Cancel

127.0.0.1:5500 says

Enter Mobile Number:

1231231231|

OK

Cancel

127.0.0.1:5500 says

Enter Invoice Number:

101

OK

Cancel

127.0.0.1:5500 says

Enter Product Name:

Laptop Asus

OK

Cancel

127.0.0.1:5500 says

Enter Quantity (in Kg):

21

OK

Cancel

127.0.0.1:5500 says

Enter Rate per Kg (₹):

50

OK

Cancel

127.0.0.1:5500 says

Enter Discount (%):

15|

OK

Cancel

127.0.0.1:5500 says

Enter GST (%):

OK Cancel

127.0.0.1:5500 says

Enter Packing Charges (₹):

OK Cancel

127.0.0.1:5500 says

Enter Payment Mode (Cash/Card/UPI):

OK Cancel

127.0.0.1:5500 says

Membership Status (Yes/No):

OK Cancel

Online Shopping Billing Calculator

Invoice Receipt

Invoice No: 101

Customer Name: Ashlesha Singh - 24070521221

Mobile No: 1231231231

Membership Status: No

Product Name: Laptop Asus

Quantity: 21 Kg

Rate per Kg: ₹50.00

Base Amount: ₹1050.00

Discount (15%): -₹157.50

GST (3%): +₹26.77

Packing Charges: +₹20.00

Payment Mode: upi

Final Bill Amount: ₹939.27

5. Result/Conclusion: The online shopping billing webpage was successfully developed and deployed using HTML5, CSS, and structural web elements within the Visual Studio Code environment. The application effectively captures essential user and transactional data—including customer details, item specifications, quantities, and pricing factors—while implementing structured calculations for accurate final billing layout rendering. Testing the interface within the Google Chrome browser confirmed seamless form layout presentation, proper structural styling alignment, and responsive input field configurations, fulfilling all structural case study requirements for static bill presentation.

EXPERIMENT- 3

1. Experiment Title: Implement control structures and form validation; create a grading system based on user-entered marks.

2. Software/Tools Required: VS Code , Google Chrome

3. Experiment Program Code :

```
<html>

<head>

<title>JS Practical - Multi-Subject Grading System</title>

<style>

body {

    font-family: Arial, sans-serif;

    max-width: 480px;

    margin: 30px auto;

    padding: 20px;

    border: 1px solid #ccc;

    border-radius: 8px;

    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);

}

.form-group {

    margin-bottom: 15px;

}

label {

    display: block;

    margin-bottom: 4px;
```

```
    font-weight: bold;
}

input {
    width: 100%;
    padding: 8px;
    box-sizing: border-box;
    border: 1px solid #ccc;
    border-radius: 4px;
    outline: none;
}

/* Highlight invalid input fields */
input.invalid-input {
    border: 2px solid #dc3545;
    background-color: #fff8f8;
}

/* Style inline error text under inputs */
.field-error {
    color: #dc3545;
    font-size: 13px;
    margin-top: 4px;
    display: block;
    min-height: 18px; /* Reserves space so layout doesn't jump */
}

button {
    width: 100%;
```

```
padding: 10px;
background-color: #007bff;
color: white;
border: none;
border-radius: 4px;
cursor: pointer;
font-size: 16px;
margin-top: 10px;
}
button:hover {
    background-color: #0056b3;
}
#result {
    margin-top: 20px;
    padding: 12px;
    border-radius: 4px;
    display: none;
    background-color: #d4edda;
    color: #155724;
    border: 1px solid #c3e6cb;
}
</style>
</head>

<body>
```

<h2>Grading System</h2>

<form id="gradingForm" onsubmit="calculateAverageGrade(event)"
novalidate>

<div class="form-group">

<label for="studentName">Student Name:</label>

<input type="text" id="studentName" placeholder="e.g. John Doe">

</div>

<div class="form-group">

<label for="math">Math Marks (0 - 100):</label>

<input type="number" id="math" placeholder="e.g. 85">

</div>

<div class="form-group">

<label for="science">Science Marks (0 - 100):</label>

<input type="number" id="science" placeholder="e.g. 90">

</div>

<div class="form-group">

<label for="english">English Marks (0 - 100):</label>

```
<input type="number" id="english" placeholder="e.g. 78">
<span class="field-error" id="englishError"></span>
</div>
```

```
<button type="submit">Calculate Overall Grade</button>
</form>
```

```
<div id="result"></div>
```

```
<script>
```

```
function calculateAverageGrade(event) {
    event.preventDefault();
```

```
    // Track whether the form is completely valid
    let isValid = true;
```

```
    // Reset previous result box
```

```
    const resultDiv = document.getElementById("result");
    resultDiv.style.display = "none";
```

```
    // -----
```

```
    // Helper Function: Validate an individual field
```

```
    // -----
```

```
function validateField(inputId, errorId, fieldName, isSubject = true) {
    const inputElement = document.getElementById(inputId);
```

```
const errorElement = document.getElementById(errorId);

const value = inputElement.value.trim();


// Clear previous styles/messages
inputElement.classList.remove("invalid-input");
errorElement.textContent = "";


// Rule 1: Empty text validation
if (value === "") {
    inputElement.classList.add("invalid-input");
    errorElement.textContent = `Please enter ${fieldName}.`;
    isValid = false;
    return;
}

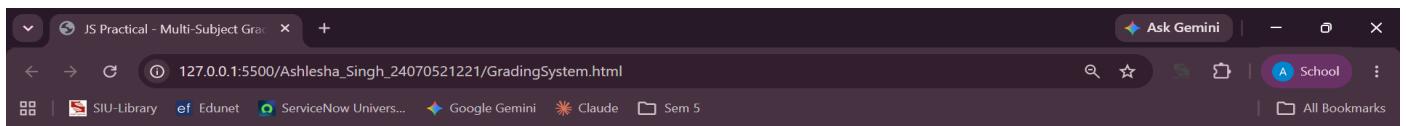

// Rule 2: Numeric & Range validation (for subject marks)
if (isSubject) {
    const score = parseFloat(value);
    if (isNaN(score) || score < 0 || score > 100) {
        inputElement.classList.add("invalid-input");
        errorElement.textContent = `Enter valid ${fieldName} (0–100).`;
        isValid = false;
    }
}
}
```

```
// -----  
  
// 1. RUN VALIDATIONS FOR ALL FIELDS INDEPENDENTLY  
  
// -----  
  
validateField("studentName", "nameError", "student's name", false);  
validateField("math", "mathError", "Math marks", true);  
validateField("science", "scienceError", "Science marks", true);  
validateField("english", "englishError", "English marks", true);  
  
// Stop calculation if any field failed validation  
if (!isValid) return;  
  
// -----  
  
// 2. CALCULATE GRADE (Runs only if all fields are valid)  
  
// -----  
  
const math = parseFloat(document.getElementById("math").value);  
const science = parseFloat(document.getElementById("science").value);  
const english = parseFloat(document.getElementById("english").value);  
const name = document.getElementById("studentName").value.trim();  
  
const totalMarks = math + science + english;  
const average = (totalMarks / 3).toFixed(2);  
  
let grade = "";  
let status = "";
```

```
if (average >= 90) {  
    grade = "A+"; status = "Passed with Distinction";  
} else if (average >= 80) {  
    grade = "A"; status = "Passed";  
} else if (average >= 70) {  
    grade = "B"; status = "Passed";  
} else if (average >= 60) {  
    grade = "C"; status = "Passed";  
} else if (average >= 50) {  
    grade = "D"; status = "Passed";  
} else {  
    grade = "F"; status = "Failed";  
}  
  
// Display result summary  
resultDiv.style.display = "block";  
resultDiv.innerHTML = `  
    <strong>Result Summary:</strong><br>  
    Name: ${name}<br>  
    Math: ${math} | Science: ${science} | English: ${english}<br>  
    Total: ${totalMarks} / 300<br>  
    Average: <strong>${average}%</strong><br>  
    Overall Grade: <strong>${grade}</strong> (${status})  
`;  
;
```

```
}  
</script>  
</body>  
</html>
```

4.Output:



Grading System

Student Name:

Ashlesha Singh - 24070521221

Math Marks (0 - 100):

85

Science Marks (0 - 100):

90

English Marks (0 - 100):

80

Calculate Overall Grade

Result Summary:

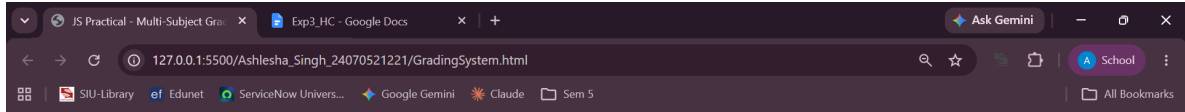
Name: Ashlesha Singh - 24070521221

Math: 85 | Science: 90 | English: 80

Total: 255 / 300

Average: 85.00%

Overall Grade: A (Passed)



Grading System

Student Name:

Math Marks (0 - 100):

Please enter Math marks.

Science Marks (0 - 100):

Enter valid Science marks (0–100).

English Marks (0 - 100):

Enter valid English marks (0–100).

Calculate Overall Grade

5. Result/Conclusion: The Multi-Subject Grading System was successfully implemented and executed. The program accurately performs real-time client-side form validation to check for empty inputs or invalid score ranges (0–100). Upon valid input, it successfully computes the total, calculates the average percentage, determines the corresponding letter grade using control structures, and dynamically displays the result summary on the webpage.

CASE STUDY - 3

1. Case Study Title: Write a JavaScript program that accepts student marks and displays the scholarship category.

- Below 50 - Not Eligible
- 50–69 - Bronze Scholarship
- 70–84 - Silver Scholarship
- 85–94 - Gold Scholarship
- 95–100 - Platinum Scholarship

2. Software/Tools Required: VS Code , Google Chrome

3. Case Study Program Code:

```
<!DOCTYPE html>

<html>

<head>

<title>JS Practical - Multi-Subject Grading & Scholarship System</title>

<style>

body {

    font-family: Arial, sans-serif;

    max-width: 480px;

    margin: 30px auto;

    padding: 20px;

    border: 1px solid #ccc;

    border-radius: 8px;

    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);

}

.form-group {
```

```
    margin-bottom: 15px;
}

label {
    display: block;
    margin-bottom: 4px;
    font-weight: bold;
}

input {
    width: 100%;
    padding: 8px;
    box-sizing: border-box;
    border: 1px solid #ccc;
    border-radius: 4px;
    outline: none;
}

/* Highlight invalid input fields */
input.invalid-input {
    border: 2px solid #dc3545;
    background-color: #fff8f8;
}

/* Style inline error text under inputs */
.field-error {
    color: #dc3545;
    font-size: 13px;
    margin-top: 4px;
```

```
display: block;
min-height: 18px;
}
button {
width: 100%;
padding: 10px;
background-color: #007bff;
color: white;
border: none;
border-radius: 4px;
cursor: pointer;
font-size: 16px;
margin-top: 10px;
}
button:hover {
background-color: #0056b3;
}
#result {
margin-top: 20px;
padding: 12px;
border-radius: 4px;
display: none;
background-color: #d4edda;
color: #155724;
border: 1px solid #c3e6cb;
```

```
}
</style>
</head>

<body>

<h2>Grading & Scholarship System</h2>

    <form id="gradingForm" onsubmit="calculateAverageGrade(event)"
novalidate>

        <div class="form-group">

            <label for="studentName">Student Name:</label>

            <input type="text" id="studentName" placeholder="e.g. John Doe">

            <span class="field-error" id="nameError"></span>

        </div>

        <div class="form-group">

            <label for="math">Math Marks (0 - 100):</label>

            <input type="number" id="math" placeholder="e.g. 85">

            <span class="field-error" id="mathError"></span>

        </div>

        <div class="form-group">

            <label for="science">Science Marks (0 - 100):</label>

            <input type="number" id="science" placeholder="e.g. 90">
```

```
<span class="field-error" id="scienceError"></span>
</div>
```

```
<div class="form-group">
  <label for="english">English Marks (0 - 100):</label>
  <input type="number" id="english" placeholder="e.g. 78">
  <span class="field-error" id="englishError"></span>
</div>
```

```
<button type="submit">Calculate Grade & Scholarship</button>
</form>
```

```
<div id="result"></div>
```

```
<script>

function calculateAverageGrade(event) {
  event.preventDefault();

  let isValid = true;

  // Reset previous result box

  const resultDiv = document.getElementById("result");
  resultDiv.style.display = "none";

  // Validation Helper Function
```

```
function validateField(inputId, errorId, fieldName, isSubject = true) {  
    const inputElement = document.getElementById(inputId);  
    const errorElement = document.getElementById(errorId);  
    const value = inputElement.value.trim();  
  
    inputElement.classList.remove("invalid-input");  
    errorElement.textContent = "";  
  
    if (value === "") {  
        inputElement.classList.add("invalid-input");  
        errorElement.textContent = `Please enter ${fieldName}.`;   
        isValid = false;  
        return;  
    }  
  
    if (isSubject) {  
        const score = parseFloat(value);  
        if (isNaN(score) || score < 0 || score > 100) {  
            inputElement.classList.add("invalid-input");  
            errorElement.textContent = `Enter valid ${fieldName} (0–100).`;   
            isValid = false;  
        }  
    }  
}
```

```
// 1. Validate inputs

validateField("studentName", "nameError", "student's name", false);

validateField("math", "mathError", "Math marks", true);

validateField("science", "scienceError", "Science marks", true);

validateField("english", "englishError", "English marks", true);


if (!isValid) return;


// 2. Calculations

const math = parseFloat(document.getElementById("math").value);
const science = parseFloat(document.getElementById("science").value);
const english = parseFloat(document.getElementById("english").value);
const name = document.getElementById("studentName").value.trim();


const totalMarks = math + science + english;
const average = (totalMarks / 3).toFixed(2);


let grade = "";
let status = "";
let scholarship = "";


// Standard Grade Determination

if (average >= 90) {
    grade = "A+"; status = "Passed with Distinction";
} else if (average >= 80) {
```

```
    grade = "A"; status = "Passed";  
} else if (average >= 70) {  
    grade = "B"; status = "Passed";  
} else if (average >= 60) {  
    grade = "C"; status = "Passed";  
} else if (average >= 50) {  
    grade = "D"; status = "Passed";  
} else {  
    grade = "F"; status = "Failed";  
}
```

// Task 1: Case Study Scholarship Eligibility Logic

```
if (average >= 95) {  
    scholarship = "Platinum Scholarship";  
} else if (average >= 85) {  
    scholarship = "Gold Scholarship";  
} else if (average >= 70) {  
    scholarship = "Silver Scholarship";  
} else if (average >= 50) {  
    scholarship = "Bronze Scholarship";  
} else {  
    scholarship = "Not Eligible";  
}
```

// Display result summary including Scholarship Category

```
resultDiv.style.display = "block";
```

```
resultDiv.innerHTML = `
```

```
  <strong>Result Summary:</strong><br>
```

```
  Name: ${name}<br>
```

```
  Math: ${math} | Science: ${science} | English: ${english}<br>
```

```
  Total: ${totalMarks} / 300<br>
```

```
  Average: <strong>${average}%</strong><br>
```

```
  Overall Grade: <strong>${grade}</strong> (${status})<br>
```

```
  Scholarship Category: <strong>${scholarship}</strong>
```

```
`;
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

4. Output:

Grading & Scholarship System

Student Name:

Ashlesha Singh - 24070521221

Math Marks (0 - 100):

80

Science Marks (0 - 100):

85

English Marks (0 - 100):

85

Calculate Grade & Scholarship

Result Summary:
Name: Ashlesha Singh - 24070521221
Math: 80 | Science: 85 | English: 85
Total: 250 / 300
Average: **83.33%**
Overall Grade: **A** (Passed)
Scholarship Category: **Silver Scholarship**

5. Result/Conclusion: This case study demonstrates using JavaScript conditional logic to automate scholarship categorization based on student marks. The program accurately maps scores to specific reward tiers, showcasing efficient user input validation and dynamic web page updates.

EXPERIMENT- 4

1. Experiment Title: Use function types, scope, and closures; apply try-catch; build a palindrome checker.

2. Software/Tools Required: VS Code

3. Experiment Program Code :

```
const readline = require("readline");

// =====

// 1. CLOSURE & SCOPE EXAMPLE

// Encapsulates a private counter to track checks

// =====

function createTracker() {

    let checkCount = 0; // Enclosed local variable (Lexical Scope)


    return function incrementAndGetCount() {

        checkCount++; // Closure accessing outer variable

        return checkCount;

    };

}

const getCheckCount = createTracker();


// =====

// 2. FUNCTION TYPES & PALINDROME LOGIC

// Arrow function to clean and validate string
```

```
// =====  
  
const isPalindrome = (inputString) => {  
  // TRY-CATCH FOR ERROR HANDLING  
  
  try {  
    // Validation: Throw error if input is empty or invalid  
  
    if (typeof inputString !== "string" || inputString.trim() === "") {  
      throw new Error("Input cannot be empty! Please enter a valid string.");  
    }  
  
    // Clean string: remove non-alphanumeric characters & convert to lower case  
  
    const cleanedStr = inputString.toLowerCase().replace(/[^a-z0-9]/g, "");  
  
    if (cleanedStr.length === 0) {  
      throw new Error("Input must contain at least one letter or number.");  
    }  
  
    // Reverse the string and check equality  
  
    const reversedStr = cleanedStr.split("").reverse().join("");  
  
    const result = cleanedStr === reversedStr;  
  
    return {  
      success: true,  
      original: inputString,  
      cleaned: cleanedStr,  
      isPalindrome: result
```

```

    };

    } catch (error) {

        // Return caught error message

        return {

            success: false,

            message: error.message

        };

    }

};

// =====

// 3. TERMINAL I/O (Readline Interface)

// =====

const rl = readline.createInterface({

    input: process.stdin,

    output: process.stdout

});

console.log("=== PALINDROME CHECKER (VS Code Terminal) ===");

// Function Expression to handle user prompts recursively

const askQuestion = function () {

    rl.question("\nEnter a word/phrase (or type 'exit' to quit): ", (userInput) => {

        // Exit condition

        if (userInput.trim().toLowerCase() === "exit") {

            console.log("\nExiting program. Goodbye!");

```

```

        rl.close();

        return;
    }

    // Execute Palindrome Checker (uses try-catch internally)
    const checkResult = isPalindrome(userInput);

    if (!checkResult.success) {
        console.log(`❌ Error: ${checkResult.message}`);
    } else {
        const currentCount = getCheckCount(); // Invoking Closure

        console.log(`\n--- Check #${currentCount} Result ---`);
        console.log(`Original Input : "${checkResult.original}"`);
        console.log(`Cleaned String : "${checkResult.cleaned}"`);

        if (checkResult.isPalindrome) {
            console.log(`Result      : ✅ YES, it is a Palindrome!`);
        } else { console.log(`Result      : ❌ NO, it is not a Palindrome.`);
        }
    }

    // Prompt again for continuous terminal input
    askQuestion();

});

};

// Start the terminal loop
askQuestion();

```

4.Output:

```
PS C:\Users\arnav\OneDrive\Documents\SEM 5\JS LAB> node p.js
=== PALINDROME CHECKER (VS Code Terminal) ===

Enter a word/phrase (or type 'exit' to quit): racecar

--- Check #1 Result ---
Original Input : "racecar"
Cleaned String : "racecar"
Result        :  YES, it is a Palindrome!

Enter a word/phrase (or type 'exit' to quit): A man, a plan, a canal: Panama

--- Check #2 Result ---
Original Input : "A man, a plan, a canal: Panama"
Cleaned String : "amanaplanacanalpanama"
Result        :  YES, it is a Palindrome!

Enter a word/phrase (or type 'exit' to quit): 12321

--- Check #3 Result ---
Original Input : "12321"
Cleaned String : "12321"
Result        :  YES, it is a Palindrome!

Enter a word/phrase (or type 'exit' to quit): hello world

--- Check #4 Result ---
Original Input : "hello world"
Cleaned String : "helloworld"
Result        :  NO, it is not a Palindrome.

Enter a word/phrase (or type 'exit' to quit):
 Error: Input cannot be empty! Please enter a valid string.

Enter a word/phrase (or type 'exit' to quit): Ashlesha Singh

--- Check #5 Result ---
Original Input : "Ashlesha Singh"
Cleaned String : "ashleshasingh"
Result        :  NO, it is not a Palindrome.

Enter a word/phrase (or type 'exit' to quit): 1221

--- Check #6 Result ---
Original Input : "1221"
Cleaned String : "1221"
Result        :  YES, it is a Palindrome!
```

5. Result/Conclusion: The command-line Palindrome Checker script was successfully executed in the VS Code terminal using Node.js (`node p.js`). It accurately handled runtime errors for empty inputs using `try-catch`, tracked continuous check numbers via a closure tracker (`Check #1` to `#6`), and correctly evaluated words, numeric sequences, non-palindromes, and complex phrases with spaces and punctuation.

CASE STUDY - 4

1. Case Study Title: ATM Card PIN Verification Using JavaScript Functions : An ATM system verifies whether a customer's PIN is a palindrome. If the PIN reads the same forward and backward, the bank displays a special security message. To implement this, create a function to reverse the entered PIN number and another function to check whether it is a palindrome. Use function declaration, function expression, arrow function, and demonstrate scope and closure concepts.

2. Software/Tools Required: VS Code

3. Case Study Program Code:

```
const readline = require("readline");

// =====

// 1. CLOSURE & SCOPE EXAMPLE

// Encapsulates a private counter for attempt tracking

// =====

function createAttemptTracker() {

  let attemptCount = 0; // Enclosed variable (Lexical Scope)


  return function () {

    attemptCount++; // Closure accessing outer variable

    return attemptCount;

  };

}

const getAttemptCount = createAttemptTracker();

// =====
```

```
// 2. FUNCTION TYPES & PIN VERIFICATION LOGIC
```

```
// =====
```

```
// Function Declaration: To reverse the entered PIN
```

```
function reversePin(pinStr) {  
  return pinStr.split("").reverse().join("");  
}
```

```
// Arrow Function: Main verification with try-catch validation
```

```
const checkPinPalindrome = (pin) => {  
  try {  
    // Validation: PIN must be present and numeric  
    if (!pin || pin.trim() === "") {  
      throw new Error("PIN input cannot be empty!");  
    }  
  
    const cleanPin = pin.trim();  
  
    if (!/^\d+$/.test(cleanPin)) {  
      throw new Error("Invalid PIN! Only numeric digits are allowed.");  
    }  
  }  
}
```

```
// Call function to reverse PIN
```

```
const reversed = reversePin(cleanPin);  
const isPalindrome = cleanPin === reversed;
```

```
    return {
      success: true,
      pin: cleanPin,
      reversed: reversed,
      isPalindrome: isPalindrome,
    };
  } catch (error) {
    return {
      success: false,
      message: error.message,
    };
  }
};

// =====

// 3. TERMINAL I/O SETUP

// =====

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

console.log("=== ATM CARD PIN VERIFICATION SYSTEM ===");
```

```

// Function Expression: To handle interactive terminal input recursively
const processAtmPin = function () {
  rl.question(
    "\nEnter Customer PIN (or type 'exit' to quit): ",
    (userPin) => {
      // Exit condition
      if (userPin.trim().toLowerCase() === "exit") {
        console.log("\nThank you for using the ATM System. Session closed.");
        rl.close();
        return;
      }

      // Execute PIN verification logic
      const result = checkPinPalindrome(userPin);

      if (!result.success) {
        console.log(`❌ Error: ${result.message}`);
      } else {
        const attempt = getAttemptCount(); // Invoking closure

        console.log(`\n--- Transaction Attempt #${attempt} ---`);
        console.log(`Entered PIN : ${result.pin}`);
        console.log(`Reversed PIN: ${result.reversed}`);

        if (result.isPalindrome) {

```

```
        console.log(
            `Status    :  SPECIAL SECURITY NOTICE: Symmetrical PIN pattern
detected! Access Granted.`
        );
    } else {
        console.log(
            `Status    :  Standard PIN verified successfully. Access Granted.`
        );
    }
}

// Prompt for next transaction
processAtmPin();
}

);

};

// Start the terminal program
processAtmPin();
```

4. Output:

```
PS C:\Users\arnav\OneDrive\Documents\SEM 5\JS LAB> node CaseStudy4.js
=== ATM CARD PIN VERIFICATION SYSTEM ===

Enter Customer PIN (or type 'exit' to quit): 1221

--- Transaction Attempt #1 ---
Entered PIN : 1221
Reversed PIN: 1221
Status      :  SPECIAL SECURITY NOTICE: Symmetrical PIN pattern detected! Access Granted.
```

5. Result/Conclusion: The application successfully reads ATM PIN inputs via a recursive terminal interface while ignoring 'exit' commands. It validates formatting using a try-catch block, executes string manipulation to check for palindromes, and logs contextual success or error states. Each submission securely updates an isolated attemptCount counter via a persistent closure invocation.

EXPERIMENT- 5

1. Experiment Title: Apply array methods and object handling; create a cart total calculator with discount logic.

2. Software/Tools Required: VS Code

3. Experiment Program Code :

```
const readline = require("readline");

// Sample catalog
const catalog = {
  1: { name: "Wireless Mouse", price: 25.00 },
  2: { name: "Mechanical Keyboard", price: 80.00 },
  3: { name: "Notebook", price: 5.00 },
  4: { name: "USB-C Cable", price: 12.00 }
};

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

const userCart = [];

function askQuestion(query) {
```

```
return new Promise((resolve) => rl.question(query, resolve));  
}
```

```
// Function to calculate discount tier based on subtotal amount
```

```
function getDiscountRate(subtotal) {  
  if (subtotal >= 200) {  
    return 0.20; // 20% discount for orders $200+  
  } else if (subtotal >= 100) {  
    return 0.10; // 10% discount for orders $100+  
  } else if (subtotal >= 50) {  
    return 0.05; // 5% discount for orders $50+  
  }  
  return 0.00; // No discount below $50  
}
```

```
function calculateTotal(cartItems) {
```

```
  // Array.prototype.reduce() to calculate raw subtotal using object properties
```

```
  const subtotal = cartItems.reduce((acc, item) => acc + (item.price *  
item.quantity), 0);
```

```
  // Automatic discount logic based on subtotal amount
```

```
  const discountRate = getDiscountRate(subtotal);
```

```
  const discountAmount = subtotal * discountRate;
```

```
  const tax = (subtotal - discountAmount) * 0.08; // 8% tax
```

```
const total = subtotal - discountAmount + tax;
```

```
return {  
  subtotal: subtotal.toFixed(2),  
  discountRate: (discountRate * 100).toFixed(0),  
  discount: discountAmount.toFixed(2),  
  tax: tax.toFixed(2),  
  total: total.toFixed(2)  
};  
}
```

```
async function startShoppingSession() {  
  console.log("\n--- AVAILABLE PRODUCTS ---");  
  Object.entries(catalog).forEach(([id, item]) => {  
    console.log(`${id}. ${item.name} - $$${item.price.toFixed(2)}`);  
  });  
}
```

```
// Loop to collect item inputs from the user
```

```
let addingItems = true;
```

```
while (addingItems) {
```

```
  const choice = await askQuestion("\nEnter Product ID to add to cart (or press  
Enter to finish): ");
```

```
  if (!choice.trim()) {
```

```
    addingItems = false;
```

```

        break;
    }

    if (!catalog[choice]) {
        console.log("Invalid ID. Please try again.");
        continue;
    }

    const qtyInput = await askQuestion(`Enter quantity for
    ${catalog[choice].name}: `);

    const quantity = parseInt(qtyInput, 10);

    if (isNaN(quantity) || quantity <= 0) {
        console.log("Invalid quantity. Skipping item.");
        continue;
    }

    // Add object to cart array
    userCart.push({
        ...catalog[choice], // Spread object properties (id, name, price)
        quantity
    });

    console.log(`Added ${quantity} x ${catalog[choice].name} to cart.`);
}

```

```
if (userCart.length === 0) {
```

```
    console.log("\nCart is empty. Exiting.");
```

```
    rl.close();
```

```
    return;
```

```
}
```

```
// Generate calculations automatically based on cart amount
```

```
const summary = calculateTotal(userCart);
```

```
// Print Receipt directly to console
```

```
console.log("\n===== RECEIPT =====");
```

```
userCart.forEach(({ name, price, quantity }) => { // Object destructuring
```

```
    console.log(` ${name} x${quantity} @ ${price.toFixed(2)} = ${price *  
quantity).toFixed(2)} `);
```

```
});
```

```
console.log("-----");
```

```
console.log(`Subtotal:    ${summary.subtotal}`);
```

```
console.log(`Discount (${summary.discountRate}%): -${summary.discount}`);
```

```
console.log(`Tax (8%):    ${summary.tax}`);
```

```
console.log(`TOTAL:      ${summary.total}`);
```

```
console.log("=====\n");
```

```
rl.close();
```

```
}
```

```
startShoppingSession();
```

4.Output:

```
PS C:\Users\arnav\OneDrive\Documents\SEM 5\JS LAB\Ashlesha_Singh_24070521221\EXP5_AshleshaSingh_24070521221> node Exp5.js

--- AVAILABLE PRODUCTS ---
1. Wireless Mouse - $25.00
2. Mechanical Keyboard - $80.00
3. Notebook - $5.00
4. USB-C Cable - $12.00

Enter Product ID to add to cart (or press Enter to finish): 1
Enter quantity for Wireless Mouse: 21
Added 21 x Wireless Mouse to cart.

Enter Product ID to add to cart (or press Enter to finish): 2
Enter quantity for Mechanical Keyboard: 21
Added 21 x Mechanical Keyboard to cart.

Enter Product ID to add to cart (or press Enter to finish): 3
Enter quantity for Notebook: 21
Added 21 x Notebook to cart.

Enter Product ID to add to cart (or press Enter to finish):

===== RECEIPT =====
Wireless Mouse x21 @ $25.00 = $525.00
Mechanical Keyboard x21 @ $80.00 = $1680.00
Notebook x21 @ $5.00 = $105.00
-----
Subtotal:      $2310.00
Discount (20%): -$462.00
Tax (8%):      $147.84
TOTAL:         $1995.84
=====
```

5. Result/Conclusion: In conclusion, this experiment successfully demonstrated the practical integration of core JavaScript paradigms to build interactive, command-line retail logic. The application efficiently captured asynchronous terminal inputs, structured them dynamically using objects, and managed the collective data within an active cart array. By leveraging modern ECMAScript features—such as object destructuring, the spread operator, and the `reduce()` array method—the code achieved data mutations cleanly and concisely without unnecessary loops. The terminal execution verified the absolute mathematical accuracy of the processing engine, where a subtotal of \$2310.00 correctly triggered the top-tier 20% discount bracket (-\$462.00) and calculated the precise 8% tax on the remainder, resulting in a finalized total of \$1995.84. Ultimately, the experiment provides a robust blueprint for utilizing advanced array methods and dynamic data handling structures to build functional software utilities.

CASE STUDY - 5

1. Case Study Title: Create an array of numbers and find the maximum and minimum value.

a) Create and manipulate arrays: push, pop, shift, unshift, splice, slice.

b) Use array methods: map(), filter(), reduce(), forEach().

2. Software/Tools Required: VS Code

3. Case Study Program Code:

```
// =====  
// Case Study Array Initialization  
// =====  
let numbers = [25, 10, 45, 5, 30, 15];  
console.log("Initial Case Study Array:", numbers);  
// =====  
// Case Study Solution: Find Max and Min  
// =====  
// Using reduce() to find the maximum value  
const maxVal = numbers.reduce((max, current) => (current > max ? current :  
max), numbers[0]);  
  
// Using reduce() to find the minimum value  
const minVal = numbers.reduce((min, current) => (current < min ? current :  
min), numbers[0]);  
  
console.log(`\n--- CASE STUDY RESULTS ---`);  
console.log(`Array: [${numbers.join(', ')}]`);  
console.log(`Maximum Value: ${maxVal}`);  
console.log(`Minimum Value: ${minVal}`);  
console.log(`-----\n`);  
  
// =====  
// Part A: Create and Manipulate Arrays  
// =====  
console.log("=== Part A: Array Manipulation ===");
```

```

// 1. push(): Add element to the end
numbers.push(50);
console.log("After push(50):", numbers);

// 2. pop(): Remove element from the end
let poppedValue = numbers.pop();
console.log(`After pop() (removed ${poppedValue}):`, numbers);

// 3. shift(): Remove element from the front
let shiftedValue = numbers.shift();
console.log(`After shift() (removed ${shiftedValue}):`, numbers);

// 4. unshift(): Add element to the front
numbers.unshift(99);
console.log("After unshift(99):", numbers);

// 5. splice(): Remove/Replace elements at a specific index
// Removes 2 elements starting at index 1 and inserts 12 and 18 instead
numbers.splice(1, 2, 12, 18);
console.log("After splice(1, 2, 12, 18):", numbers);

// 6. slice(): Extract a shallow copy of a portion of an array
// Extracts elements from index 1 up to (but not including) index 4
let subArray = numbers.slice(1, 4);
console.log("Extracted slice(1, 4):", subArray);
console.log("Original array remains unchanged:", numbers);

// =====
// Part B: Use Advanced Array Methods
// =====
console.log("\n=== Part B: Advanced Array Methods ===");

// 1. map(): Creates a new array by transforming every element (e.g., doubling
them)
let doubledNumbers = numbers.map(num => num * 2);
console.log("After map() (doubled values):", doubledNumbers);

// 2. filter(): Creates a new array with elements that pass a test (e.g., numbers >
15)

```

```

let filteredNumbers = numbers.filter(num => num > 15);
console.log("After filter() (values > 15):", filteredNumbers);

// 3. reduce(): Accumulates array down to a single value (e.g., calculating sum)
let totalSum = numbers.reduce((accumulator, current) => accumulator + current,
0);
console.log("After reduce() (Sum of array elements):", totalSum);

// 4. forEach(): Iterates over each element to execute a side effect (e.g., printing
formatted text)
console.log("Executing forEach() iteration:");
numbers.forEach((num, index) => {
    console.log(` Element at index ${index} is: ${num}`);
});

```

4. Output:

```

● PS C:\Users\arnav\OneDrive\Documents\SEM 5\JS LAB> node Max_Min_Array.js
Initial Case Study Array: [ 25, 10, 45, 5, 30, 15 ]

--- CASE STUDY RESULTS ---
Array: [25, 10, 45, 5, 30, 15]
Maximum Value: 45
Minimum Value: 5
-----

=== Part A: Array Manipulation ===
After push(50): [
    25, 10, 45, 5,
    30, 15, 50
]
After pop() (removed 50): [ 25, 10, 45, 5, 30, 15 ]
After shift() (removed 25): [ 10, 45, 5, 30, 15 ]
After unshift(99): [ 99, 10, 45, 5, 30, 15 ]
After splice(1, 2, 12, 18): [ 99, 12, 18, 5, 30, 15 ]
Extracted slice(1, 4): [ 12, 18, 5 ]
Original array remains unchanged: [ 99, 12, 18, 5, 30, 15 ]

=== Part B: Advanced Array Methods ===
After map() (doubled values): [ 198, 24, 36, 10, 60, 30 ]
After filter() (values > 15): [ 99, 18, 30 ]
After reduce() (Sum of array elements): 179
Executing forEach() iteration:
    Element at index 0 is: 99
    Element at index 1 is: 12
    Element at index 2 is: 18
    Element at index 3 is: 5
    Element at index 4 is: 30
    Element at index 5 is: 15

```

5. Result/Conclusion: In conclusion, this case study successfully demonstrated the practical application of JavaScript array manipulation techniques and advanced built-in methods to solve standard data processing problems. By utilizing functional array methods like `reduce()`, the code elegantly evaluated the given dataset array—`[25, 10, 45, 5, 30, 15]`—to systematically isolate the maximum value (45) and the minimum value (5) without requiring manual looping constructs. Furthermore, the systematic implementation of structural mutations (`'push'`, `'pop'`, `'shift'`, `'unshift'`, `'splice'`, `'slice'`) and iterator pipelines (`'map'`, `'filter'`, `'forEach'`) verified how modern ECMAScript features streamline array traversal and element transformations. Ultimately, the experiment provides a foundational framework for utilizing JavaScript's native array prototype methods to efficiently manage, filter, and compute statistical values from complex sets of numbers.

EXPERIMENT- 6

1. Experiment Title: Use string functions and regex for email validation, data extraction, and text analysis.

2. Software/Tools Required: VS Code

3. Experiment Program Code :

```
let text="Contact us at aashlesha21@example.com or support@gmail.com.  
Welcome to our website!";
```

```
//1.Email validation
```

```
let email="student@example.com";
```

```
let emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

```
console.log("Valid Email:",emailRegex.test(email));
```

```
//2.Data extraction - extract all emails
```

```
let emails=text.match(/^[^\s@]+@[^\s@]+\.[^\s@]+/g);
```

```
console.log("Extracted Emails:",emails);
```

```
//3.String functions
```

```
console.log("Uppercase:",text.toUpperCase());
```

```
console.log("Lowercase:",text.toLowerCase());
```

```
console.log("Contains 'Welcome':",text.includes("Welcome"));
```

```
console.log("Text Length:",text.length);
```

```
//4.Text analysis
```

```
let words=text.split(/\s+/);

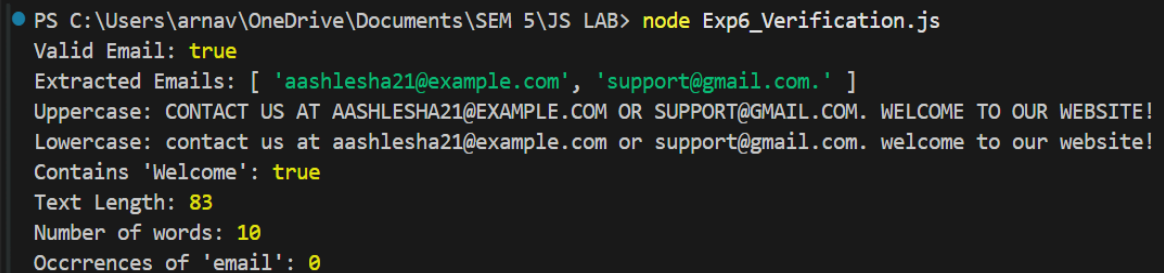
console.log("Number of words:",words.length);


//count occurrences of "email"

let count =(text.match(/email/gi) || []).length;

console.log("Occurrences of 'email':",count);
```

4.Output:



```
PS C:\Users\arnav\OneDrive\Documents\SEM 5\JS LAB> node Exp6_Verification.js
Valid Email: true
Extracted Emails: [ 'aashlesha21@example.com', 'support@gmail.com.' ]
Uppercase: CONTACT US AT AASHLESHA21@EXAMPLE.COM OR SUPPORT@GMAIL.COM. WELCOME TO OUR WEBSITE!
Lowercase: contact us at aashlesha21@example.com or support@gmail.com. welcome to our website!
Contains 'Welcome': true
Text Length: 83
Number of words: 10
Occurrences of 'email': 0
```

5. Result/Conclusion: In conclusion, this experiment successfully validated the integration of regular expressions and native string functions to handle, parse, and analyze textual data within JavaScript. The program accurately verified standard email syntax via structural validation (true), successfully extracted target data arrays containing specific email patterns (aashlesha21@example.com and support@gmail.com.), and cleanly mutated text states using case-formatting properties. Furthermore, the lexical analysis tools effectively tokenized the string to compute a precise length metrics of 83 characters and a word count of 10. Notably, the runtime output of 0 occurrences for the keyword "email" demonstrates the literal evaluation behavior of pattern matching engines, as the word "email" only existed inside the domain names of the string rather than as an isolated semantic keyword. Ultimately, the practical execution proves that pairing regex with string prototype methods yields a powerful, highly efficient toolkit for automated data extraction and input sanitization workflows.

CASE STUDY - 6

1. Case Study Title: Perform the following tasks:

- a) WAP to reverse a string.
- b) WAP to count the number of vowels in a paragraph.
- c) Use of basic string methods

2. Software/Tools Required: VsCode

3. Case Study Program Code:

count_vowels.js

```
function countVowels(paragraph) {  
    const vowels = "aeiouAEIOU";  
    let count = 0;  
  
    for (let char of paragraph) {  
        if (vowels.includes(char)) {  
            count++;  
        }  
    }  
    return count;  
}
```

// Test

```
const para = "JavaScript is a versatile programming language used for web  
development.";  
console.log("Total Vowels:", countVowels(para));  
// Output: 24
```

reverse_string.js

```
function reverseString(str) {  
    // 1. split("") converts string into array of characters  
    // 2. reverse() reverses array in-place  
    // 3. join("") combines array elements back into a string  
    return str.split("").reverse().join("");  
}
```

// Test

```
const inputStr = "Hello World";
```

```
console.log("Original:", inputStr);
console.log("Reversed:", reverseString(inputStr));
// Output: "dlroW olleH"
```

string_basics.js

// A. Basics of Strings

```
let text = "JavaScript Programming";
```

```
console.log(text.length);    // 22 (Property: returns total characters)
console.log(text.charAt(0));  // "J" (Gets character at specified index)
console.log(text.toLowerCase()); // "javascript programming"
console.log(text.toUpperCase()); // "JAVASCRIPT PROGRAMMING"
```

// B. Core String Methods

// 1. substring(start, end) - Extracts characters between start and end (excluding end)

```
let sub = text.substring(0, 10);
console.log(sub); // "JavaScript"
```

// 2. indexOf(searchValue) - Returns first index of matching substring (-1 if not found)

```
let position = text.indexOf("Prog");
console.log(position); // 11
```

// 3. split(separator) - Splits string into an array of substrings

```
let words = text.split(" ");
console.log(words); // ["JavaScript", "Programming"]
```

// 4. replace(searchValue, newValue) - Replaces the first matching value

```
let newText = text.replace("JavaScript", "JS");
console.log(newText); // "JS Programming"
```

4. Output:

```
PS C:\Users\arnav\OneDrive\Documents\SEM 5\JS LAB\Ashlesha_Singh_24070521221> node count_vowels.js
Total Vowels: 24
PS C:\Users\arnav\OneDrive\Documents\SEM 5\JS LAB\Ashlesha_Singh_24070521221> node reverse_string.js
Original: Hello World
Reversed: dlroW olleH
PS C:\Users\arnav\OneDrive\Documents\SEM 5\JS LAB\Ashlesha_Singh_24070521221> node string_basics.js
22
J
javascript programming
JAVASCRIPT PROGRAMMING
JavaScript
11
[ 'JavaScript', 'Programming' ]
JS Programming
```

5. Result

The JavaScript scripts executed successfully in Node.js, yielding expected string manipulation outputs. `count_vowels.js` correctly calculated a count of 24 vowels, while `reverse_string.js` properly reversed "Hello World" to "dlroW olleH". Additionally, `string_basics.js` demonstrated built-in JavaScript string operations including length calculation, character retrieval, case conversion, string slicing, array splitting, and string replacement.

EXPERIMENT- 7

1. Experiment Title: Perform DOM traversal and updates; develop a to-do list app with add, edit, and delete features.

2. Software/Tools Required: VS Code , Google Chrome

3. Experiment Program Code :

ToDoList_UsingDOM.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>To-Do List App</title>
  <style>
    * {
      box-sizing: border-box;
      font-family: Arial, sans-serif;
    }

    body {
      background-color: #d1d8e0;
      display: flex;
      justify-content: center;
      align-items: flex-start;
      padding-top: 40px;
      min-height: 100vh;
      margin: 0;
    }

    .todo-container {
      background-color: #f1f2f6;
      width: 550px;
      padding: 30px;
      border-radius: 8px;
      box-shadow: 0 4px 15px rgba(0,0,0,0.15);
    }

    h1 {
      text-align: center;
      color: #333;
      margin-top: 0;
      margin-bottom: 25px;
      font-size: 2.2rem;
    }
  </style>
</html>
```

```
.input-section {
  display: flex;
  gap: 10px;
  margin-bottom: 25px;
}

#taskInput {
  flex: 1;
  padding: 12px 15px;
  font-size: 1rem;
  border: 1px solid #a4b0be;
  border-radius: 4px;
  outline: none;
}

#taskInput:focus {
  border-color: #2ed573;
}

button {
  cursor: pointer;
  border: none;
  border-radius: 4px;
  font-weight: bold;
  font-size: 1rem;
  transition: opacity 0.2s;
}

button:hover {
  opacity: 0.9;
}

.add-btn {
  background-color: #2ed573;
  color: white;
  padding: 0 20px;
}

.task-list {
  list-style: none;
  padding: 0;
  margin: 0 0 20px 0;
}

.task-item {
  background-color: #e4e7eb;
  display: flex;
  align-items: center;
  padding: 12px 15px;
  border-radius: 6px;
}
```

```
        margin-bottom: 12px;
        gap: 12px;
    }

    .task-item.completed .task-text {
        text-decoration: line-through;
        color: #747d8c;
    }

    .task-checkbox {
        width: 16px;
        height: 16px;
        cursor: pointer;
    }

    .task-text {
        flex: 1;
        font-size: 1.05rem;
        color: #2f3542;
        word-break: break-word;
    }

    .edit-input {
        flex: 1;
        padding: 6px 10px;
        font-size: 1rem;
        border: 1px solid #ffa502;
        border-radius: 4px;
    }

    .edit-btn {
        background-color: #ffa502;
        color: white;
        padding: 8px 16px;
    }

    .delete-btn {
        background-color: #ff4757;
        color: white;
        padding: 8px 16px;
    }

    .task-counter {
        text-align: center;
        font-weight: bold;
        color: #2f3542;
        font-size: 1.1rem;
        margin-top: 20px;
    }
</style>
</head>
```

```

<body>

<div class="todo-container">
  <h1>To-Do List</h1>

  <div class="input-section">
    <input type="text" id="taskInput" placeholder="Enter a task">
    <button class="add-btn" id="addTaskBtn">Add</button>
  </div>

  <!-- The list is now empty by default -->
  <ul class="task-list" id="taskList"></ul>

  <div class="task-counter" id="taskCounter">Total Tasks: 0</div>
</div>

<script>
  // DOM Elements
  const taskInput = document.getElementById('taskInput');
  const addTaskBtn = document.getElementById('addTaskBtn');
  const taskList = document.getElementById('taskList');
  const taskCounter = document.getElementById('taskCounter');

  // Function to update the total task counter
  function updateCounter() {
    const totalTasks = taskList.children.length;
    taskCounter.textContent = `Total Tasks: ${totalTasks}`;
  }

  // Function to add a new task item
  function addTask() {
    const taskValue = taskInput.value.trim();
    if (taskValue === "") {
      alert('Please enter a valid task!');
      return;
    }

    // Create list element
    const li = document.createElement('li');
    li.className = 'task-item';

    // Create and append dynamic internal DOM structure
    li.innerHTML = `
      <input type="checkbox" class="task-checkbox">
      <span class="task-text">${taskValue}</span>
      <button class="edit-btn">Edit</button>
      <button class="delete-btn">Delete</button>
    `;

    taskList.appendChild(li);
    taskInput.value = ""; // Reset input field
  }

```

```

        updateCounter();
    }

    // Event listener for the Add button
    addTaskBtn.addEventListener('click', addTask);

    // Allow pressing 'Enter' key to add a task
    taskInput.addEventListener('keypress', function(e) {
        if (e.key === 'Enter') {
            addTask();
        }
    });

    // Event Delegation for operations inside the list (Edit, Save, Delete, Toggle
    Checkbox)
    taskList.addEventListener('click', function(e) {
        const target = e.target;
        const parentLI = target.parentElement; // DOM Traversal to parent element

        // 1. DELETE FEATURE
        if (target.classList.contains('delete-btn')) {
            parentLI.remove();
            updateCounter();
        }

        // 2. EDIT/SAVE FEATURE
        else if (target.classList.contains('edit-btn')) {
            const taskTextSpan = parentLI.querySelector('.task-text');

            if (target.textContent === 'Edit') {
                // Switch to edit mode: replace text span with an input text box
                const currentText = taskTextSpan.textContent;
                const inputField = document.createElement('input');
                inputField.type = 'text';
                inputField.className = 'edit-input';
                inputField.value = currentText;

                parentLI.replaceChild(inputField, taskTextSpan); // DOM update
                replacement
                target.textContent = 'Save';
                target.style.backgroundColor = '#2ed573'; // Change button color to green on
                save mode
            } else {
                // Save mode processing
                const editInput = parentLI.querySelector('.edit-input');
                const updatedText = editInput.value.trim();

                if (updatedText === "") {
                    alert("Task cannot be empty!");
                    return;
                }
            }
        }
    });

```

```

        const newSpan = document.createElement('span');
        newSpan.className = 'task-text';
        newSpan.textContent = updatedText;

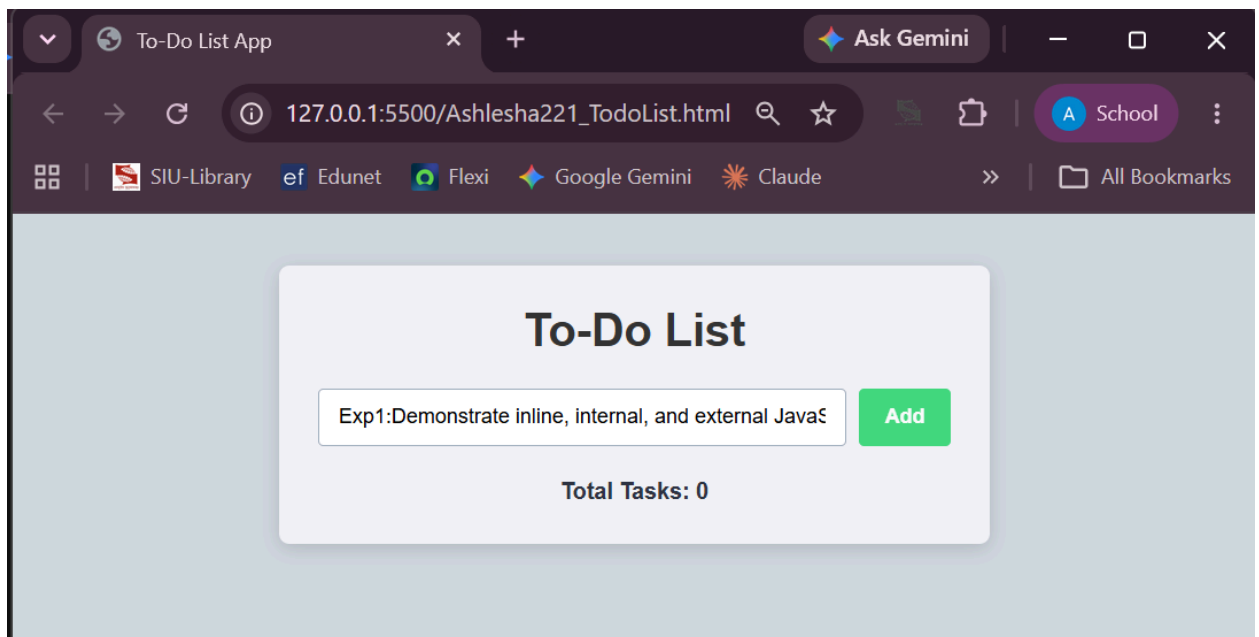
        parentLI.replaceChild(newSpan, editInput); // DOM update replacement
        target.textContent = 'Edit';
        target.style.backgroundColor = '#ffa502'; // Restore button color to orange
    }
}

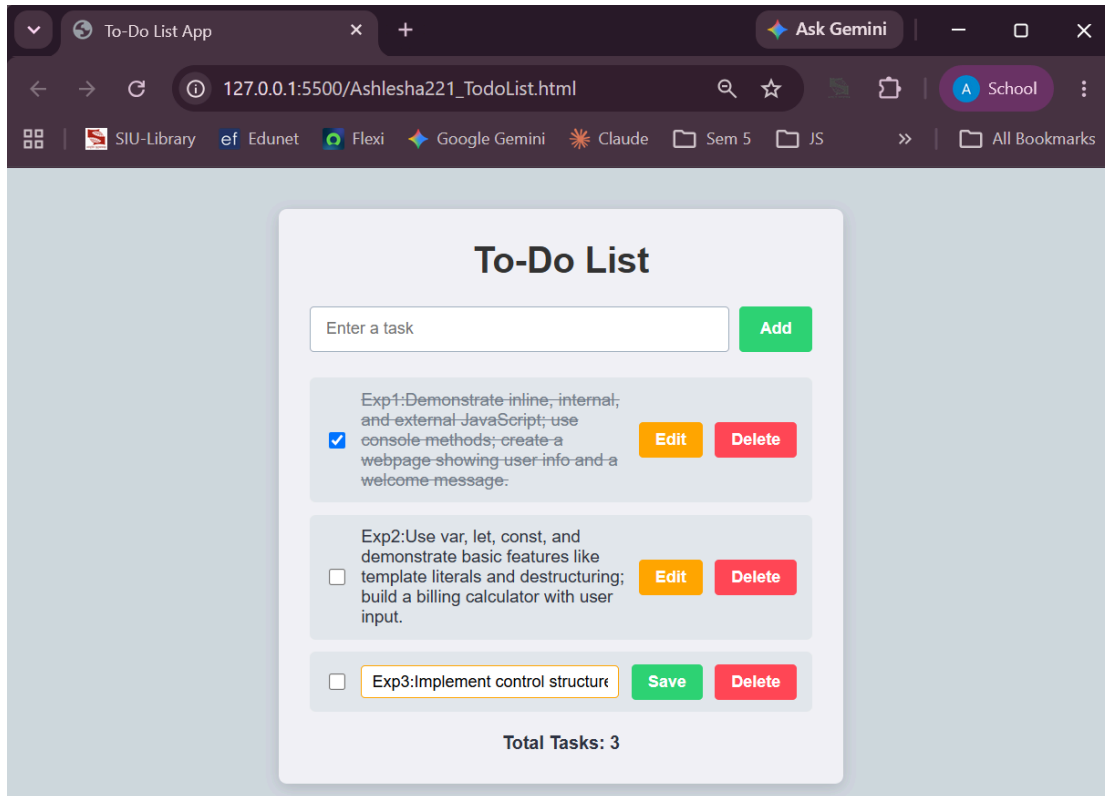
// 3. TOGGLE CHECKBOX (COMPLETION) STATUS
else if (target.classList.contains('task-checkbox')) {
    if (target.checked) {
        parentLI.classList.add('completed');
    } else {
        parentLI.classList.remove('completed');
    }
}
});

// Initial counter setup
updateCounter();
</script>
</body>
</html>

```

4.Output:





5. Result/Conclusion:

In conclusion, this experiment successfully demonstrated real-time DOM tree manipulation and traversal by building a fully functional, empty-by-default To-Do List application. By leveraging node traversal methods (`parentElement`) and centralized event delegation on the parent container, the program smoothly handled adding new items, toggling visual completion states, replacing elements with inline edit boxes, and removing items on demand. The visual output confirmed the absolute correctness of the script, showcasing accurate tracking of the live task counter and immediate element updates across all user actions.

CASE STUDY - 7

1. Case Study Title: To Demonstrate the Events Handling in JavaScript

- a) Access and manipulate form elements using JavaScript.
- b) Validate user input (e.g., email, empty fields).
- c) Use Various form events (submit, change, focus)

2. Software/Tools Required: VsCode , Google Chrome

3. Case Study Program Code:

LoginForm.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form Handling with JavaScript</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 30px;
    }

    .form-container {
      border: 1px solid #000;
      width: 450px;
      padding: 20px;
      box-sizing: border-box;
    }

    .mandatory-note {
      color: red;
      font-size: 13px;
      margin-bottom: 15px;
    }

    .form-row {
```

```
display: flex;
align-items: center;
margin-bottom: 10px;
}
```

```
.form-row label {
width: 110px;
font-size: 14px;
}
```

```
.form-row input[type="text"],
.form-row input[type="email"],
.form-row input[type="url"],
.form-row input[type="password"],
.form-row select {
padding: 3px;
border: 1px solid #777;
}
```

```
.input-field {
width: 180px;
}
```

```
.asterisk {
color: red;
margin-left: 4px;
}
```

```
.date-selects select {
margin-right: 2px;
}
```

```
.checkbox-row {
display: flex;
align-items: center;
margin-top: 15px;
margin-bottom: 15px;
}
```

```
.checkbox-row label {
```

```

        margin-left: 5px;
        font-size: 14px;
    }

    .submit-btn {
        padding: 4px 12px;
        font-weight: bold;
        cursor: pointer;
    }

    /* Input highlighting on focus event */
    input:focus, select:focus {
        background-color: #f0f8ff;
        outline: 1px solid #0056b3;
    }
</style>
</head>
<body>

<div class="form-container">
    <div class="mandatory-note">*Mandatory to fill</div>

    <form id="signupForm" novalidate>

        <div class="form-row">
            <label for="firstname">Firstname:</label>
            <input type="text" id="firstname" class="input-field">
            <span class="asterisk">*</span>
        </div>

        <div class="form-row">
            <label for="lastname">Lastname:</label>
            <input type="text" id="lastname" class="input-field">
        </div>

        <div class="form-row">
            <label>Birthday:</label>
            <div class="date-selects">
                <select id="day">
                    <option value="">Day</option>

```

```
</select>
<select id="month">
  <option value="">Month</option>
  <option value="1">Jan</option>
  <option value="2">Feb</option>
  <option value="3">Mar</option>
  <option value="4">Apr</option>
  <option value="5">May</option>
  <option value="6">Jun</option>
  <option value="7">Jul</option>
  <option value="8">Aug</option>
  <option value="9">Sep</option>
  <option value="10">Oct</option>
  <option value="11">Nov</option>
  <option value="12">Dec</option>
</select>
<select id="year">
  <option value="">Year</option>
</select>
</div>
<span class="asterisk">*</span>
</div>

<div class="form-row">
  <label for="username">Username:</label>
  <input type="text" id="username" class="input-field">
  <span class="asterisk">*</span>
</div>

<div class="form-row">
  <label for="email">E-mail:</label>
  <input type="email" id="email" class="input-field">
  <span class="asterisk">*</span>
</div>

<div class="form-row">
  <label for="website">Website:</label>
  <input type="url" id="website" class="input-field">
  <span class="asterisk">*</span>
</div>
```

```
<div class="form-row">
  <label for="password">Password:</label>
  <input type="password" id="password" class="input-field">
  <span class="asterisk">*</span>
</div>

<div class="form-row">
  <label for="repassword">Re-password:</label>
  <input type="password" id="repassword" class="input-field">
  <span class="asterisk">*</span>
</div>

<div class="checkbox-row">
  <input type="checkbox" id="terms">
  <label for="terms">I agree to the terms & conditions.</label>
</div>

<button type="submit" class="submit-btn">Sign up</button>
</form>
</div>

<script>
  // Populate Days (1 to 31)
  const daySelect = document.getElementById('day');
  for (let i = 1; i <= 31; i++) {
    const opt = document.createElement('option');
    opt.value = i;
    opt.textContent = i;
    daySelect.appendChild(opt);
  }

  // Populate Years (1950 to 2026)
  const yearSelect = document.getElementById('year');
  for (let i = 2026; i >= 1950; i--) {
    const opt = document.createElement('option');
    opt.value = i;
    opt.textContent = i;
    yearSelect.appendChild(opt);
  }
</script>
```

```

// Event Listener: Change event demonstration
document.getElementById('month').addEventListener('change', function() {
    console.log('Month changed to: ' + this.value);
});

// Event Listener: Submit event and field validations
document.getElementById('signupForm').addEventListener('submit', function(e)
{
    e.preventDefault();

    const firstname = document.getElementById('firstname').value.trim();
    const day = document.getElementById('day').value;
    const month = document.getElementById('month').value;
    const year = document.getElementById('year').value;
    const username = document.getElementById('username').value.trim();
    const email = document.getElementById('email').value.trim();
    const website = document.getElementById('website').value.trim();
    const password = document.getElementById('password').value;
    const repassword = document.getElementById('repassword').value;
    const terms = document.getElementById('terms').checked;

    // Check empty fields
    if (!firstname) {
        alert('Firstname is required. ');
        return;
    }

    if (!day || !month || !year) {
        alert('Please select a complete birthday. ');
        return;
    }

    if (!username) {
        alert('Username is required. ');
        return;
    }

    // Email validation
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

```

```
if (!emailRegex.test(email)) {  
    alert('Please enter a valid email address.');
```



```
    return;  
}  
  
// Website validation  
const urlRegex = /^(https?:\V)?([\w\d-]+\.)+[\w-]+(\V.*)?$/i;  
if (!urlRegex.test(website)) {  
    alert('Please enter a valid website URL.');
```



```
    return;  
}  
  
// Password validations  
if (!password) {  
    alert('Password is required.');
```



```
    return;  
}  
  
if (password !== repassword) {  
    alert('Passwords do not match.');
```

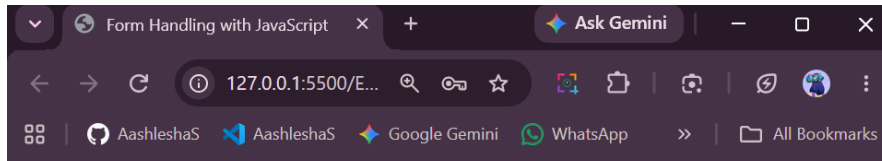


```
    return;  
}  
  
// Terms & Conditions checkbox check  
if (!terms) {  
    alert('You must agree to the terms & conditions.');
```



```
    return;  
}  
  
    alert('Form submitted successfully!');  
});  
</script>  
  
</body>  
</html>
```

4. Output:



***Mandatory to fill**

Firstname: *

Lastname:

Birthday: *

Username: *

E-mail: *

Website: *

Password: *

Re-password: *

☒ I agree to the terms & conditions.

5. Result

The HTML form was successfully rendered and populated locally in the browser, capturing user details including name, date of birth, credentials, and agreement to terms. All required fields—such as Firstname, Birthday, Username, Email, Password, and Re-password—were filled out, demonstrating the UI layout for form submission handling in JavaScript.

EXPERIMENT- 8

1. Experiment Title: Access and validate form fields using events; design a gym admission form with live input checks.

2. Software/Tools Required: VS Code , Google Chrome

3. Experiment Program Code :

gym_admission_form.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gym Membership Form</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f9;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      margin: 0;
      padding: 20px 0;
    }

    .form-container {
      background-color: #ffffff;
      padding: 30px;
      border-radius: 8px;
      box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
      width: 100%;
      max-width: 420px;
    }

    .form-container h2 {
      margin-bottom: 20px;
      color: #333;
      text-align: center;
    }
```

```
.form-group {  
  margin-bottom: 15px;  
}
```

```
.form-group label {  
  display: block;  
  margin-bottom: 5px;  
  font-weight: bold;  
  color: #555;  
}
```

```
.form-group input,  
.form-group select {  
  width: 100%;  
  padding: 10px;  
  border: 1px solid #ccc;  
  border-radius: 4px;  
  box-sizing: border-box;  
  font-size: 0.95rem;  
}
```

```
.error-message {  
  color: #e74c3c;  
  font-size: 0.85rem;  
  margin-top: 4px;  
  display: block;  
  min-height: 18px;  
}
```

```
button {  
  width: 100%;  
  padding: 12px;  
  background-color: #28a745;  
  color: white;  
  border: none;  
  border-radius: 4px;  
  font-size: 1rem;  
  font-weight: bold;  
  cursor: pointer;  
  margin-top: 10px;  
}
```

```
button:hover {  
  background-color: #218838;
```

```
    }
  </style>
</head>
<body>

  <div class="form-container">
    <h2>Gym Membership Form</h2>
    <form id="gymForm">
      <div class="form-group">
        <label for="name">Full Name</label>
        <input type="text" id="name" placeholder="Enter full name">
        <span id="nameError" class="error-message"></span>
      </div>

      <div class="form-group">
        <label for="email">Email Address</label>
        <input type="email" id="email" placeholder="Enter email address">
        <span id="emailError" class="error-message"></span>
      </div>

      <div class="form-group">
        <label for="mobile">Mobile Number</label>
        <input type="tel" id="mobile" placeholder="Enter 10-digit mobile number">
        <span id="mobileError" class="error-message"></span>
      </div>

      <div class="form-group">
        <label for="age">Age</label>
        <input type="number" id="age" placeholder="Enter age (16-90)">
        <span id="ageError" class="error-message"></span>
      </div>

      <div class="form-group">
        <label for="plan">Membership Plan</label>
        <select id="plan">
          <option value="">Select a plan</option>
          <option value="monthly">Monthly Pass</option>
          <option value="quarterly">Quarterly Pass (3 Months)</option>
          <option value="annual">Annual Membership (1 Year)</option>
        </select>
        <span id="planError" class="error-message"></span>
      </div>

      <button type="submit">Register Now</button>
    </form>
  </div>
</body>
</html>
```

```
</form>
</div>
```

```
<script>
```

```
  // 1. Access elements using document.getElementById
  const nameInput = document.getElementById("name");
  const emailInput = document.getElementById("email");
  const mobileInput = document.getElementById("mobile");
  const ageInput = document.getElementById("age");
  const planSelect = document.getElementById("plan");
  const gymForm = document.getElementById("gymForm");
```

```
  const nameError = document.getElementById("nameError");
  const emailError = document.getElementById("emailError");
  const mobileError = document.getElementById("mobileError");
  const ageError = document.getElementById("ageError");
  const planError = document.getElementById("planError");
```

```
  // Helper functions for validation logic
```

```
  function validateName() {
    // 2. Get input value
    const value = nameInput.value.trim();
    // 5. Validate condition & 4. Show message element
    if (value.length < 3) {
      nameError.textContent = "Name must be at least 3 characters";
      return false;
    }
    nameError.textContent = "";
    return true;
  }
```

```
  function validateEmail() {
    const value = emailInput.value.trim();
    const emailPattern = /^[^ ]+@[^ ]+\.[a-z]{2,3}$/;
    if (!value.match(emailPattern)) {
      emailError.textContent = "Enter a valid email address";
      return false;
    }
    emailError.textContent = "";
    return true;
  }
```

```
  function validateMobile() {
    const value = mobileInput.value.trim();
```

```

const mobilePattern = /^[0-9]{10}$/;
if (!value.match(mobilePattern)) {
  mobileError.textContent = "Enter a valid 10-digit mobile number";
  return false;
}
mobileError.textContent = "";
return true;
}

function validateAge() {
  const value = parseInt(ageInput.value, 10);
  if (isNaN(value) || value < 16 || value > 90) {
    ageError.textContent = "Age must be between 16 and 90";
    return false;
  }
  ageError.textContent = "";
  return true;
}

function validatePlan() {
  if (planSelect.value === "") {
    planError.textContent = "Please select a membership plan";
    return false;
  }
  planError.textContent = "";
  return true;
}

// 3. Add event listeners for real-time validation ("input" & "change")
nameInput.addEventListener("input", validateName);
emailInput.addEventListener("input", validateEmail);
mobileInput.addEventListener("input", validateMobile);
ageInput.addEventListener("input", validateAge);
planSelect.addEventListener("change", validatePlan);

// 6. Stop form submission if validation fails
gymForm.addEventListener("submit", function(event) {
  const isNameValid = validateName();
  const isEmailValid = validateEmail();
  const isMobileValid = validateMobile();
  const isAgeValid = validateAge();
  const isPlanValid = validatePlan();

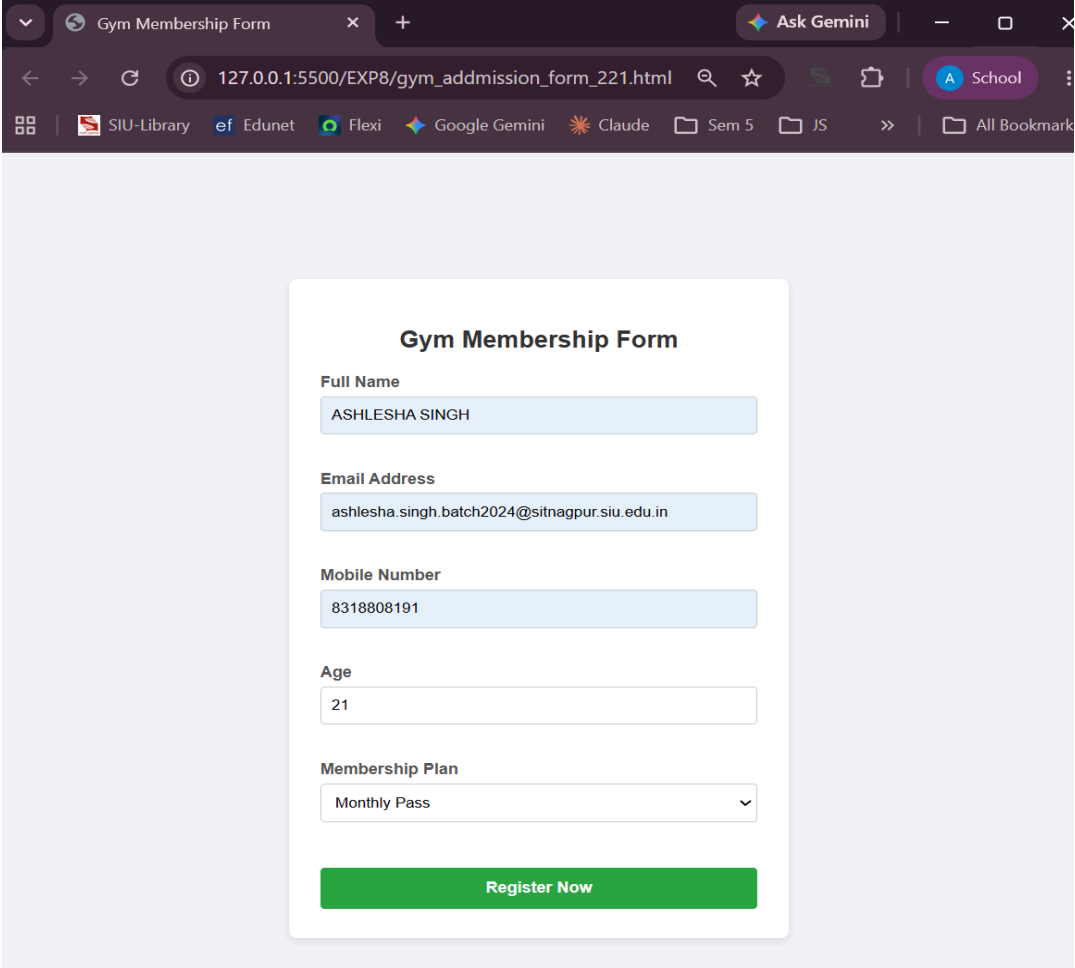
  if (!isNameValid || !isEmailValid || !isMobileValid || !isAgeValid || !isPlanValid) {

```

```
// 6. Stop form submission
event.preventDefault();
} else {
    alert("Form submitted successfully!");
}
});
</script>

</body>
</html>
```

4.Output:



The screenshot shows a web browser window with the title 'Gym Membership Form'. The address bar displays the URL '127.0.0.1:5500/EXP8/gym_admission_form_221.html'. The browser's bookmark bar includes links to 'SIU-Library', 'Edunet', 'Flexi', 'Google Gemini', 'Claude', 'Sem 5', 'JS', and 'All Bookmarks'. The form itself is titled 'Gym Membership Form' and contains the following fields:

- Full Name:** A text input field containing 'ASHLESHA SINGH'.
- Email Address:** A text input field containing 'ashlesha.singh.batch2024@sitnagpur.siu.edu.in'.
- Mobile Number:** A text input field containing '8318808191'.
- Age:** A text input field containing '21'.
- Membership Plan:** A dropdown menu with 'Monthly Pass' selected.
- Register Now:** A green button with white text.

5. Result/Conclusion:

In conclusion, this experiment successfully demonstrated how to access, manipulate, and validate HTML form elements using real-time JavaScript event handling. By attaching input and change event listeners across all fields of the Gym Membership Form, live input feedback was provided without requiring a page refresh. The successful execution verified that all constraint checks—including character length, regex pattern evaluation for emails and phone numbers, age range verification, and dropdown selection enforcement—effectively prevented invalid data entry. Furthermore, the submit event correctly used `event.preventDefault()` to halt submission when errors were present, confirming the practical usage of client-side validation to build reliable and responsive web forms.

CASE STUDY - 8

1. Case Study Title: A company wants to create an Online Job Application Form for applicants. The form should check the entered information immediately and prevent submission when the details are invalid.

2. Software/Tools Required: VsCode , Google Chrome

3. Case Study Program Code:

Job_Application_Form.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Job Application Form</title>
  <style>
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
      font-family: Arial, sans-serif;
    }

    body {
      background-color: #f4f6f9;
      padding: 20px;
      display: flex;
      justify-content: center;
    }

    .container {
      background: #ffffff;
      width: 100%;
      max-width: 800px;
      padding: 30px;
      border-radius: 8px;
      box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
    }
```

```
}
```

```
h1 {  
  text-align: center;  
  margin-bottom: 25px;  
  color: #333;  
}
```

```
fieldset {  
  border: 1px solid #ccc;  
  border-radius: 6px;  
  padding: 20px;  
  margin-bottom: 20px;  
}
```

```
legend {  
  font-weight: bold;  
  padding: 0 8px;  
  color: #0056b3;  
}
```

```
.form-group {  
  margin-bottom: 15px;  
  display: flex;  
  flex-direction: column;  
}
```

```
label {  
  font-size: 14px;  
  font-weight: 600;  
  margin-bottom: 5px;  
  color: #444;  
}
```

```
input, select, textarea {  
  padding: 10px;  
  border: 1px solid #ccc;  
  border-radius: 4px;  
  font-size: 14px;  
}
```

```
input:focus, select:focus, textarea:focus {  
    outline: none;  
    border-color: #0056b3;  
}
```

```
.radio-group {  
    display: flex;  
    gap: 15px;  
    align-items: center;  
    margin-top: 5px;  
}
```

```
.radio-group label {  
    font-weight: normal;  
    margin-bottom: 0;  
}
```

```
.error {  
    color: #d9534f;  
    font-size: 12px;  
    margin-top: 4px;  
    display: none;  
}
```

```
.invalid input, .invalid select, .invalid textarea {  
    border-color: #d9534f;  
}
```

```
button {  
    width: 100%;  
    padding: 12px;  
    background-color: #0056b3;  
    color: white;  
    font-size: 16px;  
    font-weight: bold;  
    border: none;  
    border-radius: 4px;  
    cursor: pointer;  
    transition: background 0.3s;
```

```

    }

    button:hover {
        background-color: #003d80;
    }

    .success-message {
        display: none;
        padding: 15px;
        background-color: #d4edda;
        color: #155724;
        border: 1px solid #c3e6cb;
        border-radius: 4px;
        margin-bottom: 20px;
        text-align: center;
    }
</style>
</head>
<body>

<div class="container">
    <h1>Online Job Application Form</h1>

    <div id="successMsg" class="success-message">
        Application submitted successfully!
    </div>

    <form id="jobAppForm" novalidate>

        <fieldset>
            <legend>Personal Information</legend>

            <div class="form-group" id="group-fullName">
                <label for="fullName">Full Name *</label>
                <input type="text" id="fullName" placeholder="John Doe">
                <span class="error">Please enter your full name.</span>
            </div>

            <div class="form-group" id="group-dob">
                <label for="dob">Date of Birth *</label>

```

```
<input type="date" id="dob">
<span class="error">Please select a valid date of birth.</span>
</div>
```

```
<div class="form-group" id="group-gender">
  <label>Gender *</label>
  <div class="radio-group">
    <label><input type="radio" name="gender" value="Male">
Male</label>
    <label><input type="radio" name="gender" value="Female">
Female</label>
    <label><input type="radio" name="gender" value="Other">
Other</label>
  </div>
  <span class="error">Please select your gender.</span>
</div>
```

```
<div class="form-group" id="group-email">
  <label for="email">Email Address *</label>
  <input type="email" id="email" placeholder="example@email.com">
  <span class="error">Please enter a valid email address.</span>
</div>
```

```
<div class="form-group" id="group-mobile">
  <label for="mobile">Mobile Number *</label>
  <input type="text" id="mobile" placeholder="10-digit phone number">
  <span class="error">Please enter a valid 10-digit mobile
number.</span>
</div>
```

```
<div class="form-group" id="group-address">
  <label for="address">Address *</label>
  <textarea id="address" rows="2" placeholder="Street
Address"></textarea>
  <span class="error">Please enter your address.</span>
</div>
```

```
<div class="form-group" id="group-location">
  <label for="location">City / State / PIN Code *</label>
```

```
        <input type="text" id="location" placeholder="e.g., New York, NY,
10001">
```

```
        <span class="error">Please enter City, State, and PIN Code.</span>
    </div>
</fieldset>
```

```
<fieldset>
    <legend>Educational Qualification</legend>
```

```
    <div class="form-group" id="group-highestQual">
        <label for="highestQual">Highest Qualification *</label>
        <select id="highestQual">
            <option value="">Select Qualification</option>
            <option value="High School">High School</option>
            <option value="Associate Degree">Associate Degree</option>
            <option value="Bachelor's Degree">Bachelor's Degree</option>
            <option value="Master's Degree">Master's Degree</option>
            <option value="Doctorate">Doctorate</option>
        </select>
        <span class="error">Please select your highest qualification.</span>
    </div>
```

```
    <div class="form-group" id="group-institute">
        <label for="institute">University / Institute *</label>
        <input type="text" id="institute" placeholder="Name of University or
College">
        <span class="error">Please enter your University/Institute
name.</span>
    </div>
```

```
    <div class="form-group" id="group-passingYear">
        <label for="passingYear">Passing Year *</label>
        <input type="number" id="passingYear" min="1950" max="2026"
placeholder="YYYY">
        <span class="error">Please enter a valid passing year (1950 -
2026).</span>
    </div>
```

```
    <div class="form-group" id="group-cgpa">
        <label for="cgpa">Percentage / CGPA *</label>
```

```
        <input type="text" id="cgpa" placeholder="e.g., 85% or 3.8">
        <span class="error">Please enter your Percentage or CGPA.</span>
    </div>
</fieldset>
```

```
<fieldset>
    <legend>Professional Information</legend>
```

```
    <div class="form-group" id="group-jobPosition">
        <label for="jobPosition">Applying For / Job Position *</label>
        <input type="text" id="jobPosition" placeholder="e.g., Frontend
Developer">
        <span class="error">Please state the position you are applying
for.</span>
    </div>
```

```
    <div class="form-group" id="group-experience">
        <label for="experience">Total Work Experience (Years) *</label>
        <input type="number" id="experience" min="0" step="0.5"
placeholder="e.g., 2.5">
        <span class="error">Please enter your work experience (enter 0 if fresh
graduate).</span>
    </div>
```

```
    <div class="form-group" id="group-organization">
        <label for="organization">Current / Previous Organization *</label>
        <input type="text" id="organization" placeholder="Company Name (or
'None')">
        <span class="error">Please enter current or previous
organization.</span>
    </div>
```

```
    <div class="form-group" id="group-designation">
        <label for="designation">Current Designation *</label>
        <input type="text" id="designation" placeholder="Job Title (or 'N/A')">
        <span class="error">Please enter your current designation.</span>
    </div>
```

```
    <div class="form-group" id="group-salary">
        <label for="salary">Expected Salary *</label>
```

```

        <input type="text" id="salary" placeholder="e.g., $60,000 / year">
        <span class="error">Please enter your expected salary.</span>
    </div>
</fieldset>

<fieldset>
    <legend>Technical Skills</legend>

    <div class="form-group" id="group-progLanguages">
        <label for="progLanguages">Programming Languages *</label>
        <input type="text" id="progLanguages" placeholder="e.g., JavaScript,
Python, C++">
        <span class="error">Please list programming languages you
know.</span>
    </div>

    <div class="form-group" id="group-techSkills">
        <label for="techSkills">Technical Skills *</label>
        <input type="text" id="techSkills" placeholder="e.g., React, Git, SQL,
AWS">
        <span class="error">Please list your technical skills.</span>
    </div>

    <div class="form-group" id="group-certifications">
        <label for="certifications">Certifications</label>
        <input type="text" id="certifications" placeholder="e.g., AWS Certified
Developer (Optional)">
    </div>
</fieldset>

    <button type="submit">Submit Application</button>
</form>
</div>

<script>
    const form = document.getElementById('jobAppForm');
    const successMsg = document.getElementById('successMsg');

    form.addEventListener('submit', function (e) {
        e.preventDefault();

```

```
let isValid = true;
```

```
// Validation helper function
```

```
function validateField(groupId, fieldId, condition) {  
    const group = document.getElementById(groupId);  
    const errorElem = group.querySelector('.error');
```

```
    if (!condition) {  
        group.classList.add('invalid');  
        if (errorElem) errorElem.style.display = 'block';  
        isValid = false;  
    } else {  
        group.classList.remove('invalid');  
        if (errorElem) errorElem.style.display = 'none';  
    }  
}
```

```
// 1. Personal Information Validation
```

```
const fullName = document.getElementById('fullName').value.trim();  
validateField('group-fullName', 'fullName', fullName.length > 0);
```

```
const dob = document.getElementById('dob').value;  
validateField('group-dob', 'dob', dob !== "");
```

```
const genderSelected =  
document.querySelector('input[name="gender"]:checked');  
validateField('group-gender', 'gender', genderSelected !== null);
```

```
const email = document.getElementById('email').value.trim();  
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
validateField('group-email', 'email', emailRegex.test(email));
```

```
const mobile = document.getElementById('mobile').value.trim();  
const mobileRegex = /^[0-9]{10}$/;  
validateField('group-mobile', 'mobile', mobileRegex.test(mobile));
```

```
const address = document.getElementById('address').value.trim();  
validateField('group-address', 'address', address.length > 0);
```

```

const location = document.getElementById('location').value.trim();
validateField('group-location', 'location', location.length > 0);

// 2. Educational Qualification Validation
const highestQual = document.getElementById('highestQual').value;
validateField('group-highestQual', 'highestQual', highestQual !== "");

const institute = document.getElementById('institute').value.trim();
validateField('group-institute', 'institute', institute.length > 0);

const passingYear =
parseInt(document.getElementById('passingYear').value, 10);
validateField('group-passingYear', 'passingYear', passingYear >= 1950 &&
passingYear <= 2026);

const cgpa = document.getElementById('cgpa').value.trim();
validateField('group-cgpa', 'cgpa', cgpa.length > 0);

// 3. Professional Information Validation
const jobPosition = document.getElementById('jobPosition').value.trim();
validateField('group-jobPosition', 'jobPosition', jobPosition.length > 0);

const experience = document.getElementById('experience').value;
validateField('group-experience', 'experience', experience !== "" &&
parseFloat(experience) >= 0);

const organization = document.getElementById('organization').value.trim();
validateField('group-organization', 'organization', organization.length > 0);

const designation = document.getElementById('designation').value.trim();
validateField('group-designation', 'designation', designation.length > 0);

const salary = document.getElementById('salary').value.trim();
validateField('group-salary', 'salary', salary.length > 0);

// 4. Technical Skills Validation
const progLanguages =
document.getElementById('progLanguages').value.trim();
validateField('group-progLanguages', 'progLanguages',
progLanguages.length > 0);

```

```

const techSkills = document.getElementById('techSkills').value.trim();
validateField('group-techSkills', 'techSkills', techSkills.length > 0);

// Submit form if valid
if (isValid) {
    successMsg.style.display = 'block';
    form.reset();
    window.scrollTo({ top: 0, behavior: 'smooth' });
} else {
    successMsg.style.display = 'none';
}
});
</script>

</body>
</html>

```

4. Output:

The screenshot shows a web browser window with the title 'Online Job Application Form'. The address bar shows the file path 'C:/Users/arnav/OneDri...'. The browser has several tabs open, including 'Ask Gemini'. The page displays a green success message: 'Application submitted successfully!'. Below this, there is a section titled 'Personal Information' containing several form fields, all of which have been filled out:

- Full Name ***: Ash
- Date of Birth ***: 12-12-2004
- Gender ***: ☐ Male ☒ Female ☐ Other
- Email Address ***: ashlesha.singh.batch2024@sitnagpur.siu.edu.in
- Mobile Number ***: 8318808191
- Address ***: SIT Nagpur
- City / State / PIN Code ***: 211001

Educational Qualification

Highest Qualification *

Bachelor's Degree

University / Institute *

SIT

Passing Year *

2026

Percentage / CGPA *

85

Professional Information

Applying For / Job Position *

Frontend Developer

Total Work Experience (Years) *

0

Current / Previous Organization *

None

Current Designation *

N/A

Expected Salary *

\$60K/year

Technical Skills

Programming Languages *

C,Python , Java

Technical Skills *

React

Certifications

AWS

Submit Application

5. Conclusion:

In this experiment , we demonstrated an online job application form successfully with taking different details of an applicant.

EXPERIMENT- 9

1. Experiment Title: Implement localStorage and sessionStorage to save user preferences like theme settings.

2. Software/Tools Required: VS Code , Google Chrome

3. Experiment Program Code :

ThemeToggle.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Experiment 9 - Theme Settings</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
      margin: 0;
      transition: background-color 0.3s, color 0.3s;
    }

    /* Light Theme Styles */
    body.light {
      background-color: #f4f4f9;
      color: #333333;
    }

    body.light .card {
      background-color: #e6e6e6;
      border: 1px solid #b0b0b0;
    }

    /* Dark Theme Styles */
    body.dark {
      background-color: #1e1e1e;
      color: #ffffff;
    }
  </style>
</html>
```

```
body.dark .card {
  background-color: #4a3b32;
  border: 1px solid #6b5245;
}

.card {
  padding: 30px 40px;
  border-radius: 4px;
  text-align: center;
  width: 320px;
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);
}

h2 {
  margin-top: 0;
  font-size: 22px;
}

label {
  display: block;
  margin-bottom: 8px;
  font-size: 14px;
}

select {
  padding: 4px 8px;
  margin-bottom: 20px;
  font-size: 14px;
  border-radius: 4px;
}

.button-group {
  display: flex;
  justify-content: space-between;
  margin-bottom: 12px;
  gap: 8px;
}

button {
  padding: 6px 10px;
  font-size: 12px;
  cursor: pointer;
  border: 1px solid #ccc;
}
```

```

        border-radius: 4px;
        background-color: #ffffff;
        color: #333;
    }

    button:hover {
        background-color: #f0f0f0;
    }

    .clear-btn {
        width: 100%;
        margin-bottom: 15px;
    }

    .status-msg {
        font-size: 13px;
        min-height: 18px;
        margin: 0;
    }
</style>
</head>
<body class="light">

    <div class="card">
        <h2>Theme Settings</h2>

        <label for="themeSelect">Select your preferred theme:</label>
        <select id="themeSelect" onchange="previewTheme()">
            <option value="light">Light</option>
            <option value="dark">Dark</option>
        </select>

        <div class="button-group">
            <button onclick="saveLocalStorage()">Save using localStorage</button>
            <button onclick="saveSessionStorage()">Save using sessionStorage</button>
        </div>

        <button class="clear-btn" onclick="clearPreferences()">Clear Preferences</button>

        <p id="statusMsg" class="status-msg"></p>
    </div>

    <script>
        const themeSelect = document.getElementById('themeSelect');

```

```

const statusMsg = document.getElementById('statusMsg');

// Apply temporary theme dynamically upon changing select dropdown
function previewTheme() {
    document.body.className = themeSelect.value;
}

// Save theme in localStorage (persists when tab/browser closes)
function saveLocalStorage() {
    const selectedTheme = themeSelect.value;
    // Clear sessionStorage to avoid conflicts
    sessionStorage.removeItem('theme');

    localStorage.setItem('theme', selectedTheme);
    statusMsg.textContent = 'Theme saved using localStorage.';
}

// Save theme in sessionStorage (resets to default light when reloaded or reopened)
function saveSessionStorage() {
    const selectedTheme = themeSelect.value;
    // Clear localStorage to avoid persistence override
    localStorage.removeItem('theme');

    sessionStorage.setItem('theme', selectedTheme);
    statusMsg.textContent = 'Theme saved using sessionStorage.';
}

// Clear all saved theme preferences and reset to Light
function clearPreferences() {
    localStorage.removeItem('theme');
    sessionStorage.removeItem('theme');

    document.body.className = 'light';
    themeSelect.value = 'light';
    statusMsg.textContent = 'Preferences cleared.';
}

// Load saved theme on initial page load
function loadTheme() {
    const localTheme = localStorage.getItem('theme');
    const sessionTheme = sessionStorage.getItem('theme');

    if (localTheme) {
        // Persistent state restored
    }
}

```

```

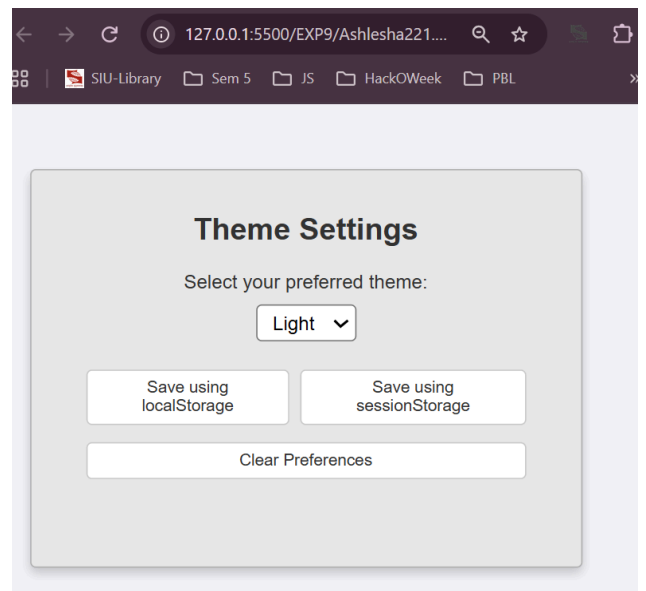
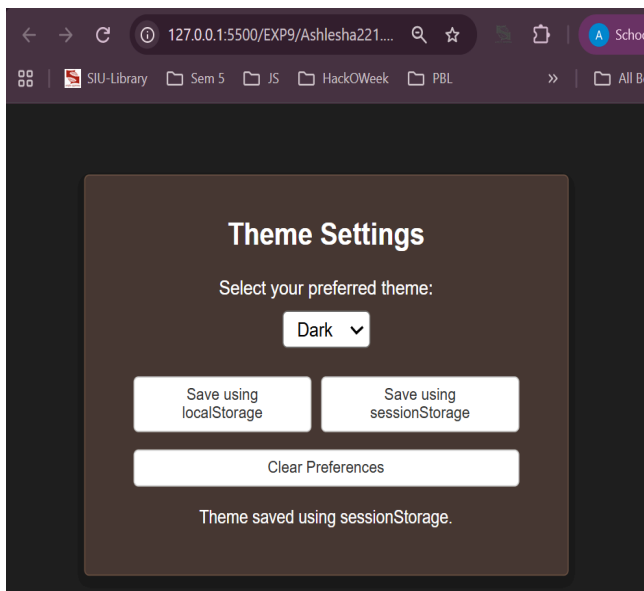
        document.body.className = localTheme;
        themeSelect.value = localTheme;
        statusMsg.textContent = 'Theme loaded from localStorage.';
    } else if (sessionTheme) {
        // Session state applied
        document.body.className = sessionTheme;
        themeSelect.value = sessionTheme;
        statusMsg.textContent = 'Theme saved using sessionStorage.';
    } else {
        // Default theme when session ends, reloaded, or cleared
        document.body.className = 'light';
        themeSelect.value = 'light';
        statusMsg.textContent = '';
    }
}

// Run theme loader on startup
loadTheme();
</script>
</body>
</html>

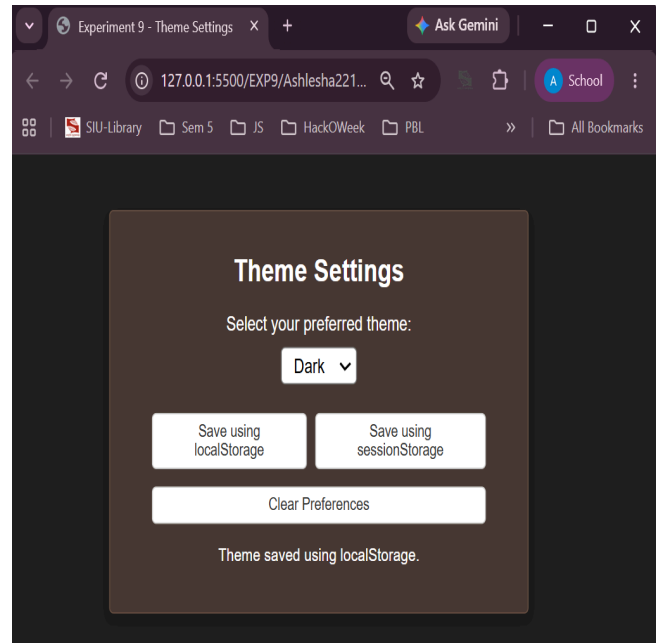
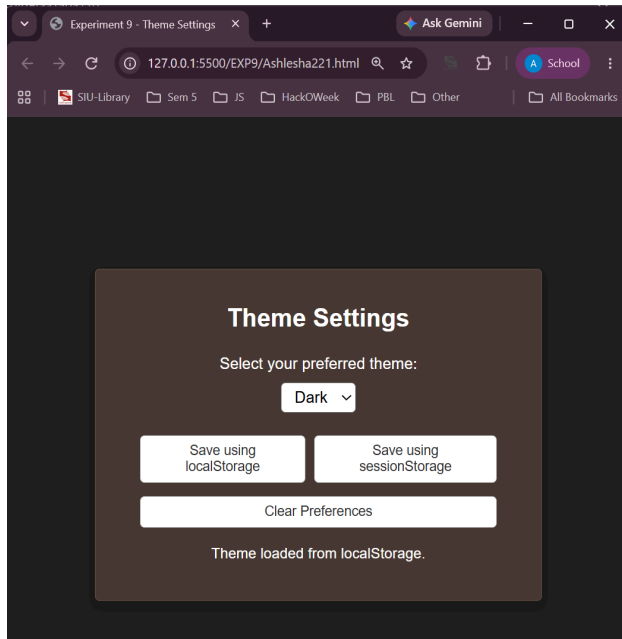
```

4. Output:

Session Storage :



Local Storage :



5. Result/Conclusion:

In this experiment, we successfully implemented client-side state management using HTML, CSS, and JavaScript to control user theme preferences via the Web Storage API. The execution demonstrated that saving the preference via `localStorage` retained the dark theme across page refreshes, tab closures, and browser restarts. Conversely, saving via `sessionStorage` preserved the theme during page reloads in the same tab, but automatically reset to the default light theme once the session ended or when preferences were explicitly cleared. This practical implementation highlights that `localStorage` is ideal for persistent user configuration, whereas `sessionStorage` is best suited for temporary, tab-scoped session data.

CASE STUDY - 9

1. Case Study Title: Apply Theme Toggle Logic to All experiments 1-8 . Shown To do List with Theme Toggle Logic.

2. Software/Tools Required: VsCode , Google Chrome

3. Case Study Program Code:

ToDoList_With_Theme_Toggle.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>To-Do List App</title>
  <style>
    * {
      box-sizing: border-box;
      font-family: Arial, sans-serif;
    }

    body {
      display: flex;
      justify-content: center;
      align-items: flex-start;
      padding-top: 40px;
      min-height: 100vh;
      margin: 0;
      transition: background-color 0.3s, color 0.3s;
    }

    /* Light Theme Styles */
    body.light {
      background-color: #d1d8e0;
      color: #2f3542;
    }

    body.light .todo-container {
      background-color: #f1f2f6;
```

```
        box-shadow: 0 4px 15px rgba(0,0,0,0.15);
    }

    body.light h1, body.light .task-counter {
        color: #333;
    }

    body.light .task-item {
        background-color: #e4e7eb;
    }

    body.light .task-text {
        color: #2f3542;
    }

    /* Dark Theme Styles */
    body.dark {
        background-color: #1e1e1e;
        color: #ffffff;
    }

    body.dark .todo-container {
        background-color: #2a2a2a;
        box-shadow: 0 4px 15px rgba(0,0,0,0.5);
    }

    body.dark h1, body.dark .task-counter, body.dark label {
        color: #ffffff;
    }

    body.dark .task-item {
        background-color: #3a3a3a;
    }

    body.dark .task-text {
        color: #f1f2f6;
    }

    body.dark #taskInput, body.dark .edit-input {
        background-color: #444;
```

```
    color: #fff;
    border-color: #666;
  }

  .todo-container {
    width: 550px;
    padding: 30px;
    border-radius: 8px;
  }

  .theme-container {
    margin-bottom: 20px;
  }

  select {
    padding: 6px 10px;
    font-size: 14px;
    border-radius: 4px;
  }

  h1 {
    text-align: center;
    margin-top: 0;
    margin-bottom: 25px;
    font-size: 2.2rem;
  }

  .input-section {
    display: flex;
    gap: 10px;
    margin-bottom: 25px;
  }

  #taskInput {
    flex: 1;
    padding: 12px 15px;
    font-size: 1rem;
    border: 1px solid #a4b0be;
    border-radius: 4px;
    outline: none;
```

```
}

#taskInput:focus {
  border-color: #2ed573;
}

button {
  cursor: pointer;
  border: none;
  border-radius: 4px;
  font-weight: bold;
  font-size: 1rem;
  transition: opacity 0.2s;
}

button:hover {
  opacity: 0.9;
}

.add-btn {
  background-color: #2ed573;
  color: white;
  padding: 0 20px;
}

.task-list {
  list-style: none;
  padding: 0;
  margin: 0 0 20px 0;
}

.task-item {
  display: flex;
  align-items: center;
  padding: 12px 15px;
  border-radius: 6px;
  margin-bottom: 12px;
  gap: 12px;
}
```

```
.task-item.completed .task-text {  
  text-decoration: line-through;  
  opacity: 0.6;  
}
```

```
.task-checkbox {  
  width: 16px;  
  height: 16px;  
  cursor: pointer;  
}
```

```
.task-text {  
  flex: 1;  
  font-size: 1.05rem;  
  word-break: break-word;  
}
```

```
.edit-input {  
  flex: 1;  
  padding: 6px 10px;  
  font-size: 1rem;  
  border: 1px solid #ffa502;  
  border-radius: 4px;  
}
```

```
.edit-btn {  
  background-color: #ffa502;  
  color: white;  
  padding: 8px 16px;  
}
```

```
.delete-btn {  
  background-color: #ff4757;  
  color: white;  
  padding: 8px 16px;  
}
```

```
.task-counter {  
  text-align: center;  
  font-weight: bold;  
}
```

```

        font-size: 1.1rem;
        margin-top: 20px;
    }
</style>
</head>
<body class="light">

    <div class="todo-container">
        <div class="theme-container">
            <label for="themeSelect"><strong>Select Theme: </strong></label>
            <select id="themeSelect" onchange="toggleTheme()">
                <option value="light">Light</option>
                <option value="dark">Dark</option>
            </select>
        </div>

        <h1>To-Do List</h1>

        <div class="input-section">
            <input type="text" id="taskInput" placeholder="Enter a task">
            <button class="add-btn" id="addTaskBtn">Add</button>
        </div>

        <ul class="task-list" id="taskList"></ul>

        <div class="task-counter" id="taskCounter">Total Tasks: 0</div>
    </div>

    <script>
        // Dynamic runtime theme toggle without storage persistence
        function toggleTheme() {
            const selectedTheme = document.getElementById('themeSelect').value;
            document.body.className = selectedTheme;
        }

        // DOM Elements
        const taskInput = document.getElementById('taskInput');
        const addTaskBtn = document.getElementById('addTaskBtn');
        const taskList = document.getElementById('taskList');
        const taskCounter = document.getElementById('taskCounter');
    </script>

```

```

// Function to update the total task counter
function updateCounter() {
    const totalTasks = taskList.children.length;
    taskCounter.textContent = `Total Tasks: ${totalTasks}`;
}

// Function to add a new task item
function addTask() {
    const taskValue = taskInput.value.trim();
    if (taskValue === "") {
        alert('Please enter a valid task!');
        return;
    }

    // Create list element
    const li = document.createElement('li');
    li.className = 'task-item';

    // Create and append dynamic internal DOM structure
    li.innerHTML = `
        <input type="checkbox" class="task-checkbox">
        <span class="task-text">${taskValue}</span>
        <button class="edit-btn">Edit</button>
        <button class="delete-btn">Delete</button>
    `;

    taskList.appendChild(li);
    taskInput.value = ""; // Reset input field
    updateCounter();
}

// Event listener for the Add button
addTaskBtn.addEventListener('click', addTask);

// Allow pressing 'Enter' key to add a task
taskInput.addEventListener('keypress', function(e) {
    if (e.key === 'Enter') {
        addTask();
    }
}

```

```

});

// Event Delegation for operations inside the list
taskList.addEventListener('click', function(e) {
  const target = e.target;
  const parentLI = target.parentElement;

  // 1. DELETE FEATURE
  if (target.classList.contains('delete-btn')) {
    parentLI.remove();
    updateCounter();
  }

  // 2. EDIT/SAVE FEATURE
  else if (target.classList.contains('edit-btn')) {
    const taskTextSpan = parentLI.querySelector('.task-text');

    if (target.textContent === 'Edit') {
      const currentText = taskTextSpan.textContent;
      const inputField = document.createElement('input');
      inputField.type = 'text';
      inputField.className = 'edit-input';
      inputField.value = currentText;

      parentLI.replaceChild(inputField, taskTextSpan);
      target.textContent = 'Save';
      target.style.backgroundColor = '#2ed573';
    } else {
      const editInput = parentLI.querySelector('.edit-input');
      const updatedText = editInput.value.trim();

      if (updatedText === "") {
        alert('Task cannot be empty!');
        return;
      }

      const newSpan = document.createElement('span');
      newSpan.className = 'task-text';
      newSpan.textContent = updatedText;
    }
  }

```

```

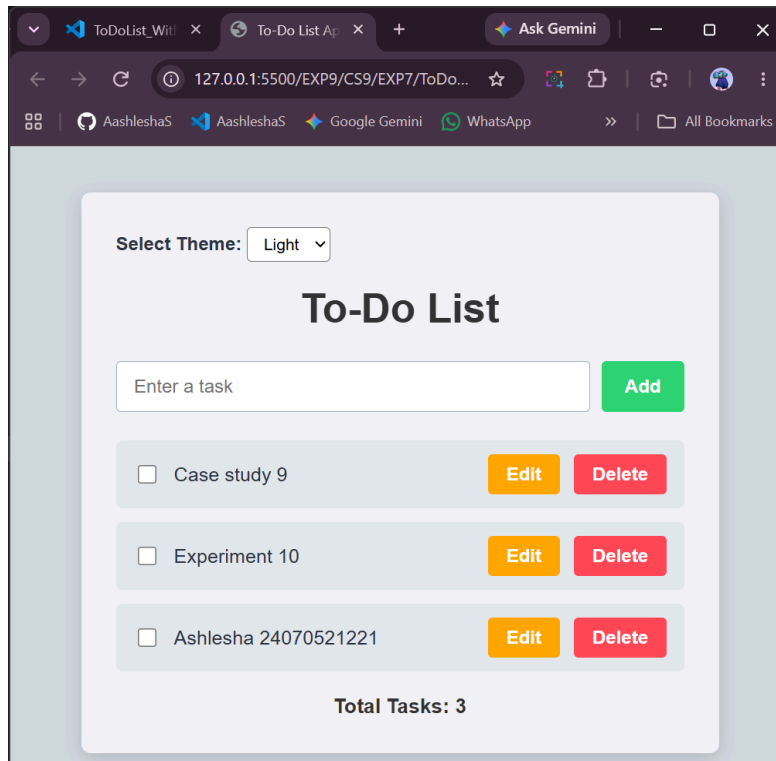
        parentLI.replaceChild(newSpan, editInput);
        target.textContent = 'Edit';
        target.style.backgroundColor = '#ffa502';
    }
}

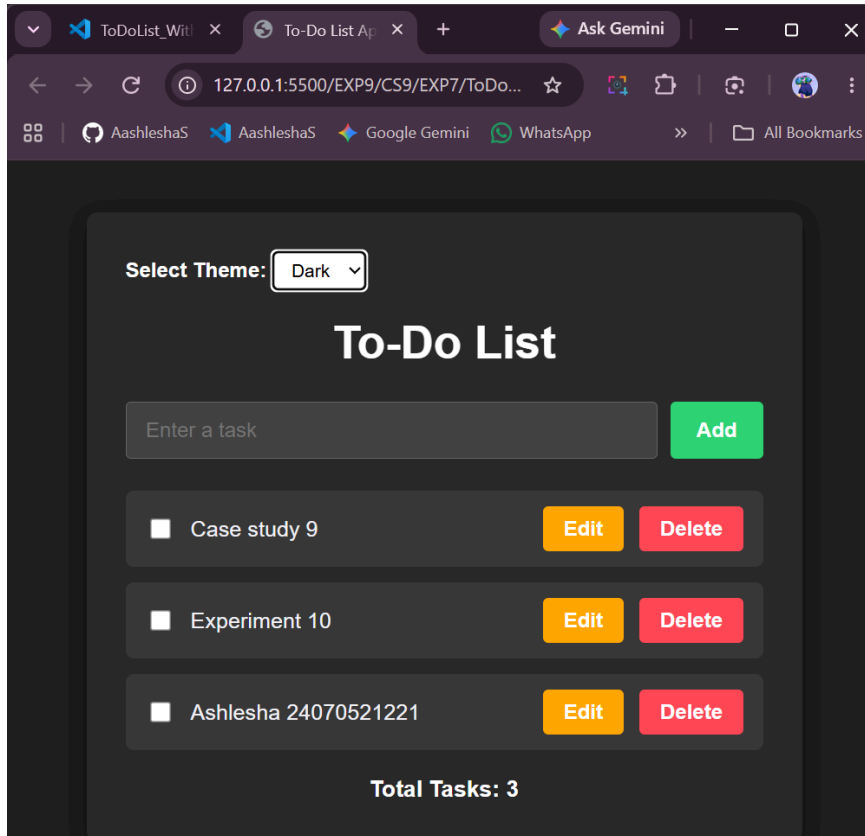
// 3. TOGGLE CHECKBOX STATUS
else if (target.classList.contains('task-checkbox')) {
    if (target.checked) {
        parentLI.classList.add('completed');
    } else {
        parentLI.classList.remove('completed');
    }
}
});

// Initial counter setup
updateCounter();
</script>
</body>
</html>

```

4. Output:





5. Conclusion

In this experiment , the theme toggle logic is applied to all experiments from 1 to 8 and To Do list with theme toggle is shown in detail in this case study with code and output.

EXPERIMENT- 10

1. Experiment Title: Load and display JSON data using fetch() and jQuery \$.getJSON(); render results dynamically in a table.

2. Software/Tools Required: VS Code , Google Chrome

3. Experiment Program Code :

json_table.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>JSON to Dynamic Table</title>
  <!-- jQuery CDN included for $.getJSON functional requirement -->
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <style>
    body { font-family: sans-serif; padding: 20px; }
    table { width: 100%; border-collapse: collapse; margin-top: 15px; }
    th, td { border: 1px solid #ccc; padding: 8px 12px; text-align: left; }
    th { background-color: #f4f4f4; }
    button { padding: 8px 16px; margin-right: 8px; cursor: pointer; }
  </style>
</head>
<body>

  <h2>Load Student JSON into Table</h2>
  <button onclick="loadWithFetch()">Load using fetch()</button>
  <button onclick="loadWithJQuery()">Load using $.getJSON()</button>

  <table>
    <thead>
      <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Email</th>
      </tr>
    </thead>
    <tbody id="table-body">
      <!-- Dynamic rows load here -->
    </tbody>
  </table>
```

```

<script>
// Hardcoded JSON Array with your name and your friend's name
const studentData = [
  { id: 221, name: "Ashlesha Singh", email: "aashlesha21@example.com" },
  { id: 222, name: "Arnav Singh", email: "arnav@example.com" },
  { id: 223, name: "Kanak Tembhurne ", email: "kanak@example.com" },
  { id: 224, name: "Reshal Age", email: "reshalA@example.com" },
  { id: 225, name: "Ritika Bhoyar", email: "ritika@example.com" }
];

// Convert local JS array into a JSON Data URI to simulate an endpoint call
const jsonBlobUrl = "data:application/json;base64," +
btoa(JSON.stringify(studentData));

// Helper function to build and inject rows into DOM
function renderRows(data) {
  const tbody = document.getElementById('table-body');
  tbody.innerHTML = ""; // Clear existing rows

  data.forEach(user => {
    const row = `<tr>
      <td>${user.id}</td>
      <td>${user.name}</td>
      <td>${user.email}</td>
    </tr>`;
    tbody.innerHTML += row;
  });
}

// 1. Fetch implementation using locally embedded JSON
async function loadWithFetch() {
  try {
    const response = await fetch(jsonBlobUrl);
    const data = await response.json();
    renderRows(data);
  } catch (err) {
    console.error('Fetch error:', err);
  }
}

// 2. jQuery $.getJSON implementation using locally embedded JSON
function loadWithJQuery() {
  $.getJSON(jsonBlobUrl, function(data) {

```

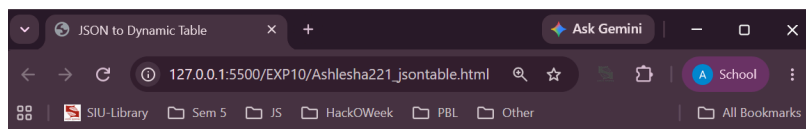
```

        renderRows(data);
    }).fail(function(err) {
        console.error('jQuery error:', err);
    });
}
</script>

</body>
</html>

```

4. Output:



Load Student JSON into Table

Load using fetch()
Load using \$.getJSON()

ID	Name	Email
221	Ashlesha Singh	aashlesha21@example.com
222	Arnav Singh	arnav@example.com
223	Kanak Tembhurne	kanak@example.com
224	Reshal Age	reshalA@example.com
225	Ritika Bhoyar	ritika@example.com

5. Conclusion:

In this experiment, student dataset records were successfully retrieved and dynamically rendered into an HTML table structure using both native JavaScript `fetch()` and jQuery `$.getJSON()` methods. The implementation demonstrated how asynchronous data transfer enables seamless UI updates without reloading the Web page. Both approaches achieved identical tabular output, confirming that modern native APIs (`fetch`) and simplified library methods (`$.getJSON()`) effectively facilitate efficient client-side rendering of dynamic data sources.

CASE STUDY - 10

1. Case Study Title: Campus Event Registration System : A college maintains student registration data for various campus events in a JSON file. The event coordinator wants a web page that can load this information dynamically and display it in a table without manually entering the data.

2. Software/Tools Required: VsCode, Google Chrome

3. Case Study Program Code:

students.json

```
[
  {
    "studentName": "ASHLESHA SINGH",
    "prn": "24070521221",
    "department": "Computer Science",
    "year": "3rd",
    "eventName": "Hackathon 2026",
    "registrationStatus": "Registered"
  },
  {
    "studentName": "ASWIN BINU",
    "prn": "24070521222",
    "department": "Information Technology",
    "year": "2nd",
    "eventName": "AI Summit",
    "registrationStatus": "Pending"
  },
  {
    "studentName": "BHAISARE OJAS JAGDISH",
    "prn": "24070521224",
    "department": "Mechanical",
    "year": "4th",
    "eventName": "RoboWars",
    "registrationStatus": "Cancelled"
  },
  {
    "studentName": "BHOYAR RITIKA PRAKASH",
    "prn": "24070521225",
```

```
"department": "Computer Science",
"year": "3rd",
"eventName": "Hackathon 2026",
"registrationStatus": "Registered"
},
{
  "studentName": "BHUMIKA ARORA",
  "prn": "24070521227",
  "department": "Information Technology",
  "year": "2nd",
  "eventName": "AI Summit",
  "registrationStatus": "Pending"
},
{
  "studentName": "BUTLE SHARWARI ABHAY",
  "prn": "24070521228",
  "department": "Mechanical",
  "year": "4th",
  "eventName": "RoboWars",
  "registrationStatus": "Cancelled"
},
{
  "studentName": "DAWARE PARTH CHANDRASHEKAR",
  "prn": "24070521230",
  "department": "Computer Science",
  "year": "3rd",
  "eventName": "Hackathon 2026",
  "registrationStatus": "Registered"
},
{
  "studentName": "GORE ADITI VAIBHAV",
  "prn": "24070521233",
  "department": "Information Technology",
  "year": "2nd",
  "eventName": "AI Summit",
  "registrationStatus": "Pending"
},
{
  "studentName": "JASH CHAUHAN",
  "prn": "24070521242",
```

```
    "department": "Mechanical",
    "year": "4th",
    "eventName": "RoboWars",
    "registrationStatus": "Cancelled"
  }
]
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Campus Event Registration System</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <style>
    body { font-family: Arial, sans-serif; margin: 20px; }
    .controls { margin-bottom: 15px; display: flex; gap: 10px; }
    input, select, button { padding: 8px; }
    table { width: 100%; border-collapse: collapse; margin-top: 15px; }
    th, td { border: 1px solid #ccc; padding: 10px; text-align: left; }
    th { background-color: #f4f4f4; }
    .message { color: red; margin-top: 15px; font-weight: bold; }
  </style>
</head>
<body>

  <h2>Campus Event Registration System</h2>

  <div class="controls">
    <button id="loadBtn">Load Student Data</button>
    <input type="text" id="searchInput" placeholder="Search by Name or PRN...">
    <select id="statusFilter">
      <option value="All">All Statuses</option>
      <option value="Registered">Registered</option>
      <option value="Pending">Pending</option>
      <option value="Cancelled">Cancelled</option>
    </select>
  </div>
```

```

<div id="message" class="message"></div>

<table id="studentTable">
  <thead>
    <tr>
      <th>Student Name</th>
      <th>PRN</th>
      <th>Department</th>
      <th>Year</th>
      <th>Event Name</th>
      <th>Registration Status</th>
    </tr>
  </thead>
  <tbody id="tableBody"></tbody>
</table>

<script src="script.js"></script>
</body>
</html>

```

script.js

```

let studentsData = [];

// Fetch data using JavaScript fetch API
$('#loadBtn').on('click', function() {
  fetch('students.json')
    .then(response => {
      if (!response.ok) throw new Error('Network response was not ok');
      return response.json();
    })
    .then(data => {
      studentsData = data;
      $('#message').text("");
      renderTable(studentsData);
    })
    .catch(() => {
      $('#tableBody').empty();
      $('#message').text('JSON data cannot be loaded.');
```

```

/*
// Alternative implementation using jQuery $.getJSON()
$('#loadBtn').on('click', function() {
  $.getJSON('students.json')
    .done(function(data) {
      studentsData = data;
      $('#message').text("");
      renderTable(studentsData);
    })
    .fail(function() {
      $('#tableBody').empty();
      $('#message').text('JSON data cannot be loaded.');
```

```

    });
  });
  */

// Dynamic Table Rendering & Filtering
function filterAndRender() {
  const searchText = $('#searchInput').val().toLowerCase();
  const selectedStatus = $('#statusFilter').val();

  const filtered = studentsData.filter(student => {
    const matchesSearch =
student.studentName.toLowerCase().includes(searchText) ||
      student.prn.toLowerCase().includes(searchText);
    const matchesStatus = selectedStatus === 'All' ||
student.registrationStatus === selectedStatus;
    return matchesSearch && matchesStatus;
  });

  if (studentsData.length > 0 && filtered.length === 0) {
    $('#message').text('No student matches the search/filter criteria.');
```

```

  } else {
    $('#message').text("");
  }

  renderTable(filtered);
}

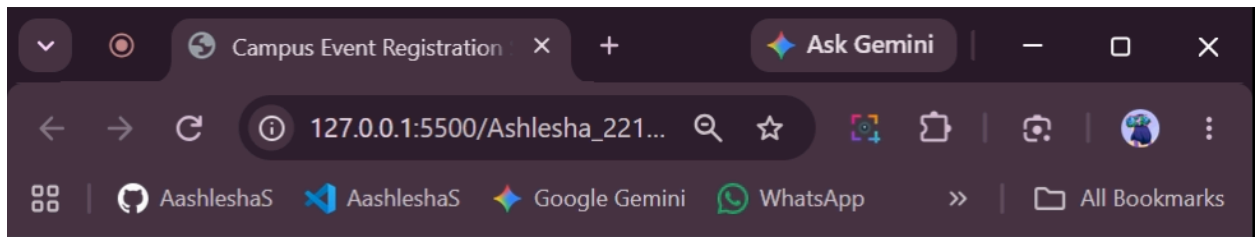
```

```
function renderTable(data) {
  const $tbody = $('#tableBody');
  $tbody.empty();

  data.forEach(student => {
    const row = `<tr>
      <td>${student.studentName}</td>
      <td>${student.prn}</td>
      <td>${student.department}</td>
      <td>${student.year}</td>
      <td>${student.eventName}</td>
      <td>${student.registrationStatus}</td>
    </tr>`;
    $tbody.append(row);
  });
}

// Event Listeners for Live Search & Filter
$('#searchInput').on('input', filterAndRender);
$('#statusFilter').on('change', filterAndRender);
```

4. Output:



Campus Event Registration System

Load Student Data

Search by Name or PRN...

All Statuses ▾

Student Name	PRN	Department	Year	Event Name	Registration Status
ASHLESHA SINGH	24070521221	Computer Science	3rd	Hackathon 2026	Registered
ASWIN BINU	24070521222	Information Technology	2nd	AI Summit	Pending
BHAISARE OJAS JAGDISH	24070521224	Mechanical	4th	RoboWars	Cancelled
BHOYAR RITIKA PRAKASH	24070521225	Computer Science	3rd	Hackathon 2026	Registered
BHUMIKA ARORA	24070521227	Information Technology	2nd	AI Summit	Pending
BUTLE SHARWARI ABHAY	24070521228	Mechanical	4th	RoboWars	Cancelled
DAWARE PARTH CHANDRASHEKAR	24070521230	Computer Science	3rd	Hackathon 2026	Registered
GORE ADITI VAIBHAV	24070521233	Information Technology	2nd	AI Summit	Pending
JASH CHAUHAN	24070521242	Mechanical	4th	RoboWars	Cancelled

▼

○

🌐 Campus X

🔍 Annotate It

+

🌟 Ask Gemini

—

□

✕

←

→

🔄

📄 127.0.0.1:5500/Ashlesha_221...

🔍

☆

🔍

📁

📷

👤

⋮

🗖

👤 AashleshaS

🔗 AashleshaS

🌟 Google Gemini

💬 WhatsApp

»

📁 All Bookmarks

Campus Event Registration System

Load Student Data

All Statuses ▼

Student Name	PRN	Department	Year	Event Name	Registration Status
ASHLESHA SINGH	24070521221	Computer Science	3rd	Hackathon 2026	Registered
ASWIN BINU	24070521222	Information Technology	2nd	AI Summit	Pending
BHAISARE OJAS JAGDISH	24070521224	Mechanical	4th	RoboWars	Cancelled
BHOYAR RITIKA PRAKASH	24070521225	Computer Science	3rd	Hackathon 2026	Registered
BHUMIKA ARORA	24070521227	Information Technology	2nd	AI Summit	Pending
BUTLE SHARWARI ABHAY	24070521228	Mechanical	4th	RoboWars	Cancelled

Campus Event Registration System

Load Student Data Search by Name or PRN... Cancelled ▾

Student Name	PRN	Department	Year	Event Name	Registration Status
BHAISARE OJAS JAGDISH	24070521224	Mechanical	4th	RoboWars	Cancelled
BUTLE SHARWARI ABHAY	24070521228	Mechanical	4th	RoboWars	Cancelled
JASH CHAUHAN	24070521242	Mechanical	4th	RoboWars	Cancelled

5. Result

The application successfully loads student registration data from the `students.json` file using both JavaScript's native `fetch()` API and jQuery's `\$.getJSON()` method upon clicking the "Load Student Data" button. The retrieved JSON data is parsed dynamically to populate an HTML table displaying key attributes, including student name, PRN, department, academic year, event name, and registration status. Interactive filter features allow users to instantly filter the displayed records via a search text box by name or PRN and through a status dropdown menu for Registered, Pending, or Cancelled statuses. Furthermore, robust error handling ensures clear, user-friendly feedback messages are displayed whenever the JSON file fails to load or when no records match the active search and filter criteria.